

Guia de clase

Técnicas digitales I

Ing. Jerónimo F. Atencio

Versión: 20

Índice

Índice	1
Acerca de esta guía	3
Modalidad de evaluación	4
Condiciones para la firma de la materia.	4
Condiciones para la promoción de la materia.	4
Realización de los parciales y recuperatorios	4
Entrega de trabajos prácticos	5
1. Herramientas de desarrollo	6
Descargando de vivado	6
Descargando e instalando vivado	8
2. Accediendo al repositorio	15
Configurando Git por primera vez	16
Clonando el repositorio de material.	16
Actualizando el la versión local del repositorio de material desde uno remoto (pull)	17
4. Algebra de boole. Minimización de funciones lógicas.	18
5. Aritmética	21
6. Circuitos combinacionales.	23
Ejemplos	23
Agregar un nuevo archivo de diseño	31
Clonando su repositorio personal.	34
Ejercicios	34
7. Generics; for - generate; componentes	38
Ejemplo	38
Ejercicios	39
8. Circuitos aritméticos	41
Ejemplos	41
Ejercicios	41
9. Circuitos secuenciales	44
Ejemplo	44
Ejercicios	45
10. Registros de desplazamiento	47
Ejercicios	47
11. Contadores binarios	49
Ejemplo	49
Ejercicios	50
12. Máquinas de estados	52
Ejercicios	52
13. Testbench	61
Ejercicios	61
14. Timing	68
15. Ejercicios integradores I.	70
Ejemplo de implementación de un LFSR	70
Ejemplos warning latch	74

Ejercicios	75
16. Ejercicios integradores II	82
17. Trabajo final, MicroIO	84
Características	84
Entidad	84
Diagrama en bloques	85
Notación	85
Set de instrucciones	86
ALU - Lógica	86
ALU - Aritmética	86
ALU - Flags	87
Jmp	87
mov	88



Acerca de esta guía

Este documento comienza a guiarte en el aprendizaje de los lineamientos básicos de diseño de sistemas digitales utilizando el lenguaje VHDL sobre Vivado, abarcando la instalación de las herramientas de desarrollo, para luego comenzar a realizar los primeros pasos en VHDL.

Cada capítulo comienza mostrando información relevante para llevarlo adelante, luego continuar con algunos ejemplos y terminar con ejercicios propuestos para realizar. Es recomendable seguir el orden de la guía y los ejercicios ya que la dificultad de los mismos va en incremento. Los ejemplos son muy básicos, pero sirven para mostrar algún aspecto del lenguaje y/o como base para la resolución de los ejercicios, por lo cual es recomendable transcribirlos y probarlos.

Se recomienda que los códigos de ejemplo los transcribas evitando copiar y pegar para ir tomando práctica en la sintaxis del lenguaje. Trate de leer la referencia del lenguaje de las funciones, sentencias y comandos utilizados para interiorizarse en su uso más allá de la aplicación particular que se haga en esta guía.

Modalidad de evaluación

La evaluación de la materia se realizará con un parcial y un trabajo final junto con un coloquio del mismo. El coloquio consistirá en 3 partes:

1. Explicación oral del código implementado y su funcionamiento. Puede ayudarse con una presentación y/o diagramas que tenga realizados.
2. Demostración de funcionamiento en simulaciones y/o kit de desarrollo
3. Evaluación escrita. Realización de modificaciones y/o mejoras al proyecto.

Condiciones para la firma de la materia.

Las condiciones de firma de la materia son:

1. Obtener 6 o más puntos en el primer parcial. Se puede recuperar en diciembre y/o en febrero.
2. Obtener 6 o más puntos en el segundo parcial. Se puede recuperar en diciembre y/o en febrero.
3. Tener aprobado el 75% de los trabajos prácticos.

Condiciones para la promoción de la materia.

Las condiciones de promoción de la materia son:

1. Obtener 8 o más puntos en el primer parcial.
2. Obtener 8 o más puntos en el segundo parcial.
3. Tener aprobado el 75% de los trabajos prácticos.

En este caso si alguno de los parciales no alcanza la condición para promoción, se podrá recuperar sólo uno en la fecha de diciembre. La nota que queda es la de la última evaluación realizada.

Ejemplos:

Parciales		Recuperatorio diciembre		Recuperatorio febrero		Trabajo práctico aprobado	Resultado
Primer parcial	Segundo parcial	Primer parcial	Segundo parcial	Primer parcial	Segundo parcial		
8	8	-	-	-	-	Si	Promociona
6	8	8	-	-	-	Si	Promociona
6	8	6	-	-	-	Si	Firma
7	7	-	5	-	6	Si	Firma
10	7	-	-	-	-	Si	Firma
10	7	-	7	-	8	Si	Firma
7	7	-	4	-	2	Si	Recurso
10	10	-	-	-	-	No	Recurso

Realización de los parciales y recuperatorios

Las fechas de los parciales y los recuperatorios las podrá encontrar en la planificación. En cada parcial entran todos los temas vistos hasta la clase anterior dicho parcial. El primer parcial y la evaluación escrita del trabajo final será presencial así como también los recuperatorios de los mismos.

Entrega de trabajos prácticos

Todos los ejercicios deberán ser subidos al repositorio en una carpeta con la siguiente convención de nombres ejercicios_AA siendo AA el número de capítulo

Por ejemplo: para el capítulo 8 la carpeta se llamará ejercicios_08

Al realizar los ejercicios salvo que indique lo contrario utilizaremos como nombre del archivo el nombre de la entidad.

Para cada capítulo se realizará un proyecto en vivo.



1.Herramientas de desarrollo

Descargando de vivado

Como herramienta para la realización de los ejercicios utilizaremos Vivado de Xilinx-AMD, se recomienda el uso de la versión 2020.2, pero a los fines de la materia cualquier versión sirve. La descarga del vivado la puede hacer desde el siguiente link.

<https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vivado-design-tools.htm>

🏠 / Adaptive Computing Support / Downloads

Downloads

Licensing Help		NIC Software & Drivers		Alveo Packages		
Vivado (HW Developer)	Vitis (SW Developer)	Vitis Embedded Platforms	PetaLinux	Power Design Manager	Device Models	Documentation Navigator

Version

2024.2
2024.1
2023.2
Vivado Archive
ISE Archive
CAE Vendor Libraries Archive

Vivado™ Edition - 2024.2 Full Product Installation

Important Information

Vivado™ 2024.2 is now available for download:

Advanced Flow for Place-and-Route of All Versal™ Devices

- Automatic partition-based placement and parallel P&R
- Reduces congestion and improves routability for fast design closure
- Default flow for all Versal devices

Enabling Top-Level RTL Flows for Versal Devices

- Configure key components like NoC and transceivers from top-level RTL
- Enables programmable logic developers to stay in a RTL-centric design environment

Download Includes

Vivado Design Suite (All Editions)

Download Type

Full Product Installation

Last Updated

Nov 18, 2024

Answers

[2024.x - Vivado Known Issues](#)

Documentation

[Release Notes](#)
[OS Support Update](#)
[What's New in Vivado](#)

Support Forums

[Installation and Licensing](#)

Una vez seleccionada la versión, le permitirá descargar web installer tanto para linux como para windows que son más livianos y descargan e instalan el vivo. La otra opción es bajar el software completo descomprimirlo e instalarlo.

 [AMD Unified Installer for FPGAs & Adaptive SoCs 2024.2: Windows Self Extracting Web Installer \(EXE - 222.91 MB\)](#)

MD5 SUM Value : 84ee3816d8cbd0e0e38a1f9ac7eaa780

Download Verification 

Digests

Signature

Public Key

 [AMD Unified Installer for FPGAs & Adaptive SoCs 2024.2: Linux Self Extracting Web Installer \(BIN - 303.93 MB\)](#)

MD5 SUM Value : 20c806793b3ea8d79273d5138fbd195f

Download Verification 

Digests

Signature

Public Key

 [AMD Unified Installer for FPGAs & Adaptive SoCs 2024.2 SFD \(TAR/GZIP - 124.81 GB\)](#)

MD5 SUM Value : 0ca31a787bbdff82b55213522e604446

Download Verification 

Digests

Signature

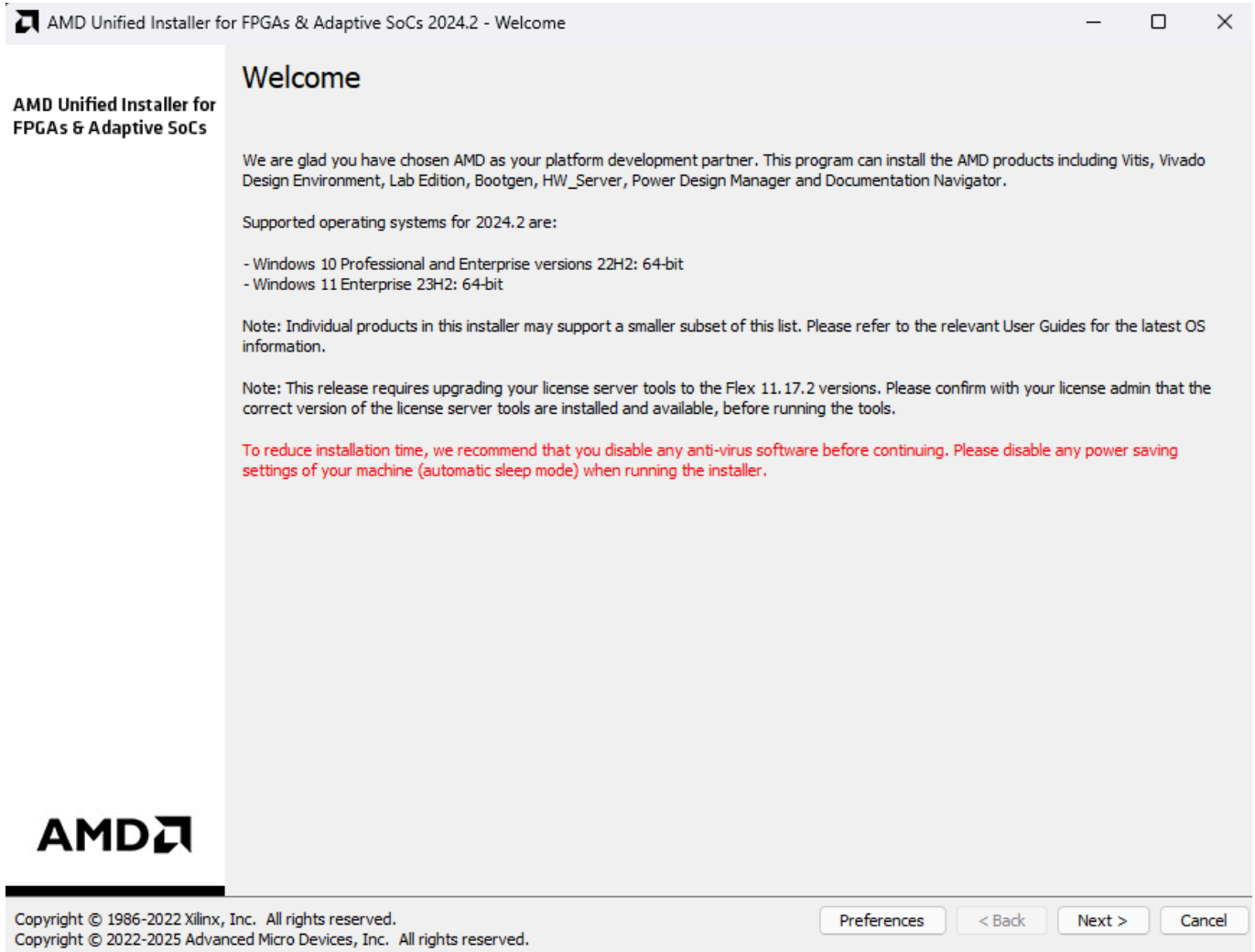
Public Key

Antes de comenzar la descarga le pedirá registrarse en la página de Xilinx-AMD.
Finalmente antes de comenzar la instalación asegúrese de tener el espacio suficiente en disco.

Descargando e instalando vivado

Descargue y ejecute la versión autoinstalable (Self Extracting Web Installer), ejecute el instalador y siga los pasos. Es recomendable desactivar el antivirus y la configuración de ahorro de energía durante la instalación.

1. Verifique los requerimientos del software y comience la instalación



2. Cree una cuenta en AMD si no posee una y luego autentiquese en la instalación

The screenshot shows a window titled "AMD Unified Installer for FPGAs & Adaptive SoCs 2024.2 - Select Install Type". The window has a dark header bar with the AMD logo on the right. Below the header, the text "Please select install type and provide your AMD.com E-mail Address and password for authentication." is displayed. The main content area is divided into two sections. The first section, "User Authentication", contains instructions to provide AMD user account credentials and links to "please create one" and "reset it here". It includes input fields for "E-mail Address" and "Password". The second section contains two radio button options: "Download and Install Now" (which is selected) and "Download Image (Install Separately)". The "Download and Install Now" option includes the instruction: "Select your desired device and tool installation options and the installer will download and install just what is required." The "Download Image (Install Separately)" option includes the instruction: "The installer will download an image containing all devices and tool options for later installation. Use this option if you wish to install a full image on a network drive or allow different users maximum flexibility when installing." At the bottom of the window, there is a footer with copyright information: "Copyright © 1986-2022 Xilinx, Inc. All rights reserved." and "Copyright © 2022-2025 Advanced Micro Devices, Inc. All rights reserved." On the right side of the footer, there are three buttons: "< Back", "Next >", and "Cancel".

AMD Unified Installer for FPGAs & Adaptive SoCs 2024.2 - Select Install Type

Select Install Type

Please select install type and provide your AMD.com E-mail Address and password for authentication.

User Authentication

Please provide your AMD user account credentials to download the required files.
If you don't have an account, [please create one](#). If you forgot your password, you can [reset it here](#).

E-mail Address

Password

☒ **Download and Install Now**

Select your desired device and tool installation options and the installer will download and install just what is required.

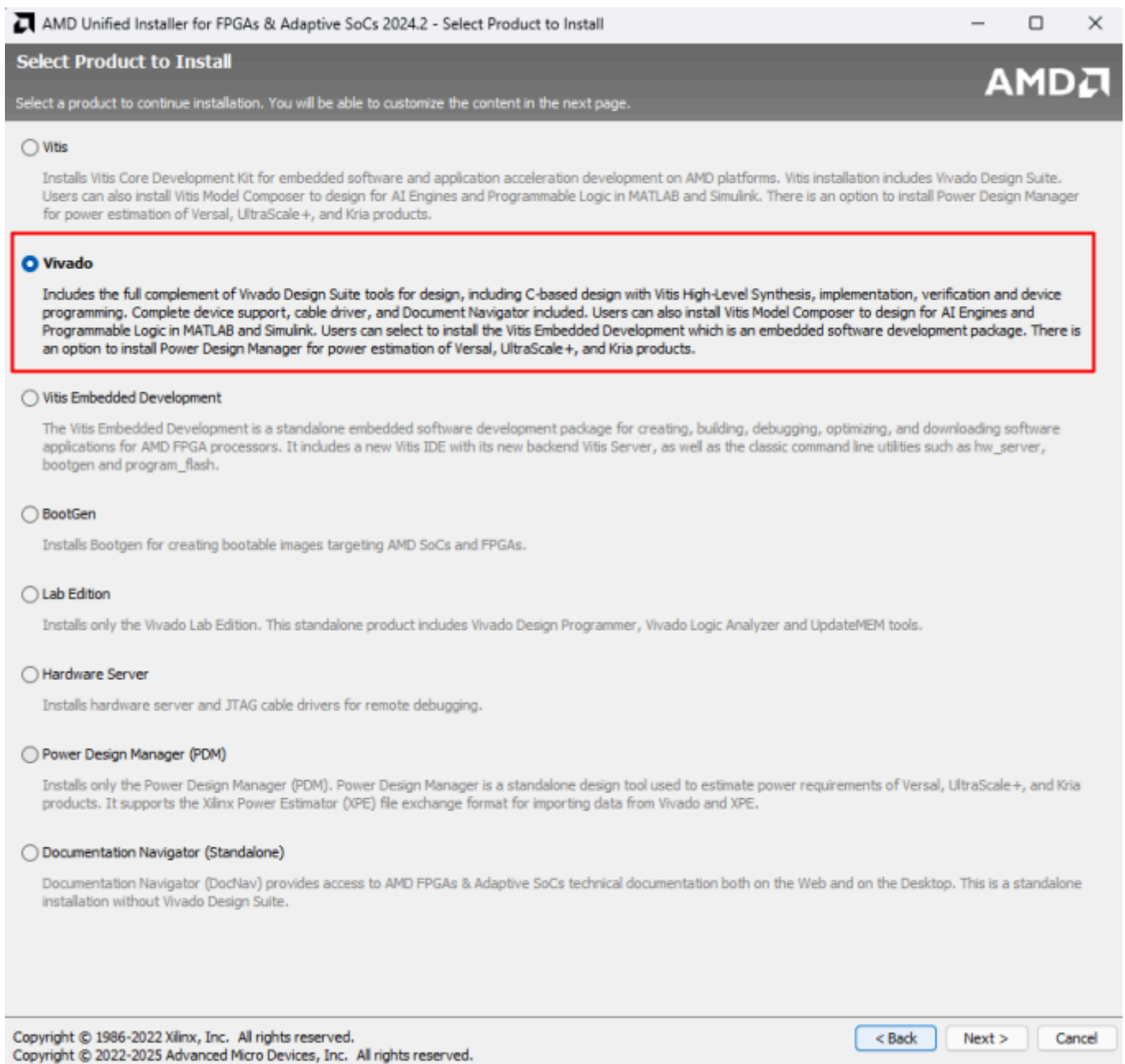
☐ **Download Image (Install Separately)**

The installer will download an image containing all devices and tool options for later installation. Use this option if you wish to install a full image on a network drive or allow different users maximum flexibility when installing.

Copyright © 1986-2022 Xilinx, Inc. All rights reserved.
Copyright © 2022-2025 Advanced Micro Devices, Inc. All rights reserved.

< Back Next > Cancel

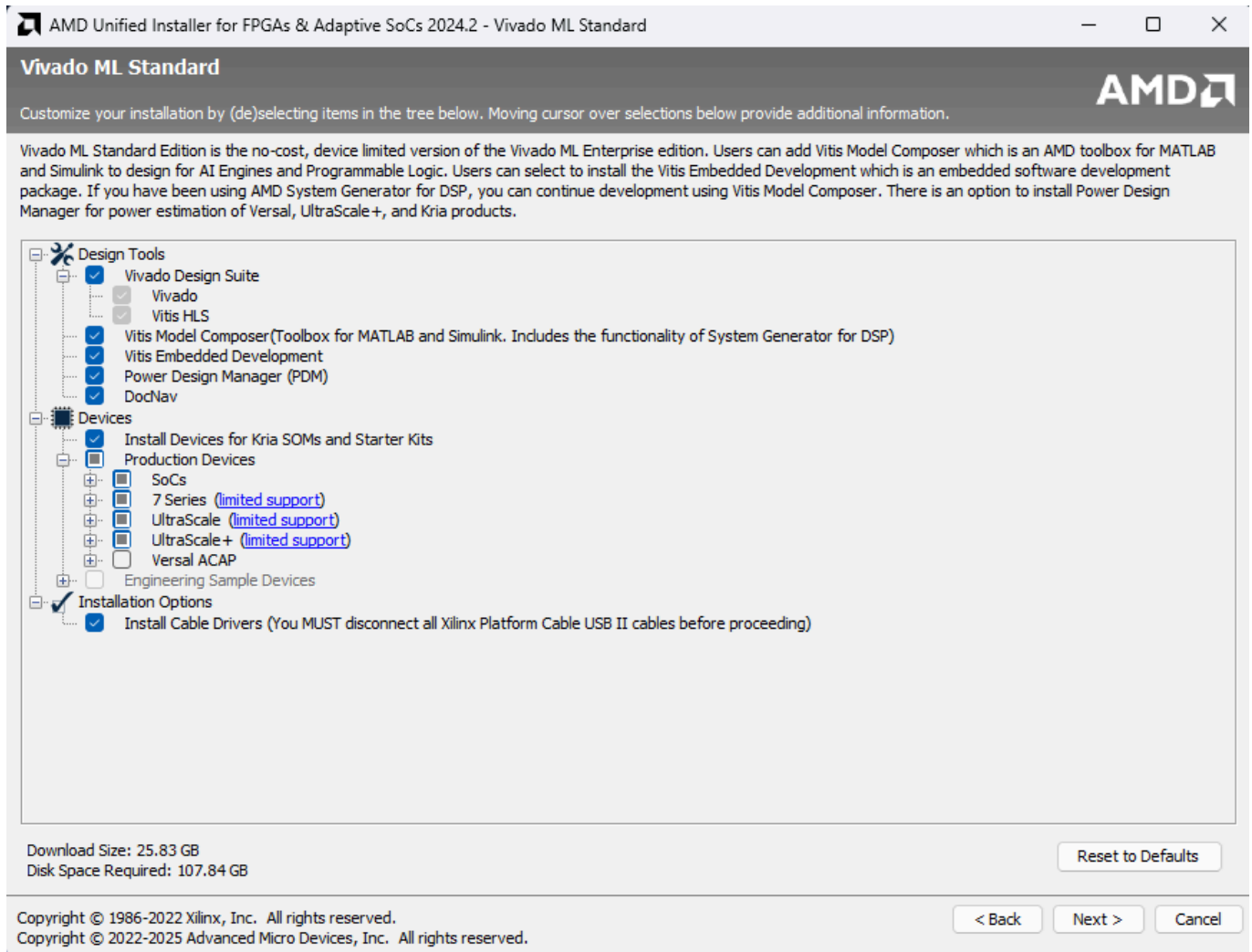
3. Seleccione el producto que instalará, en este caso Vivado



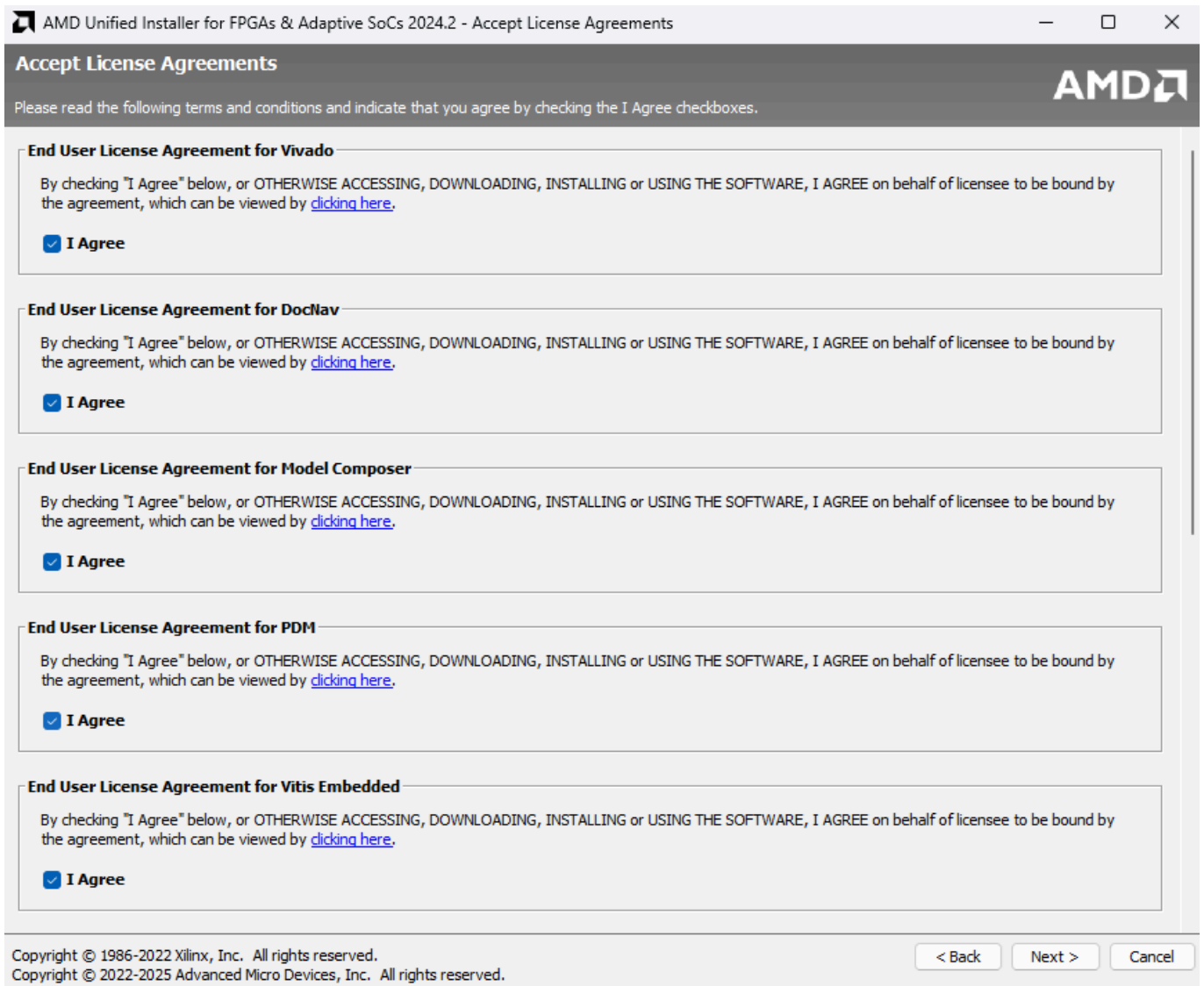
4. Seleccione la versión Standard de Vivado



5. Personalice la instalación, seleccione los dispositivos a instalar y las herramientas de desarrollo



6. Acepte los términos de la licencia

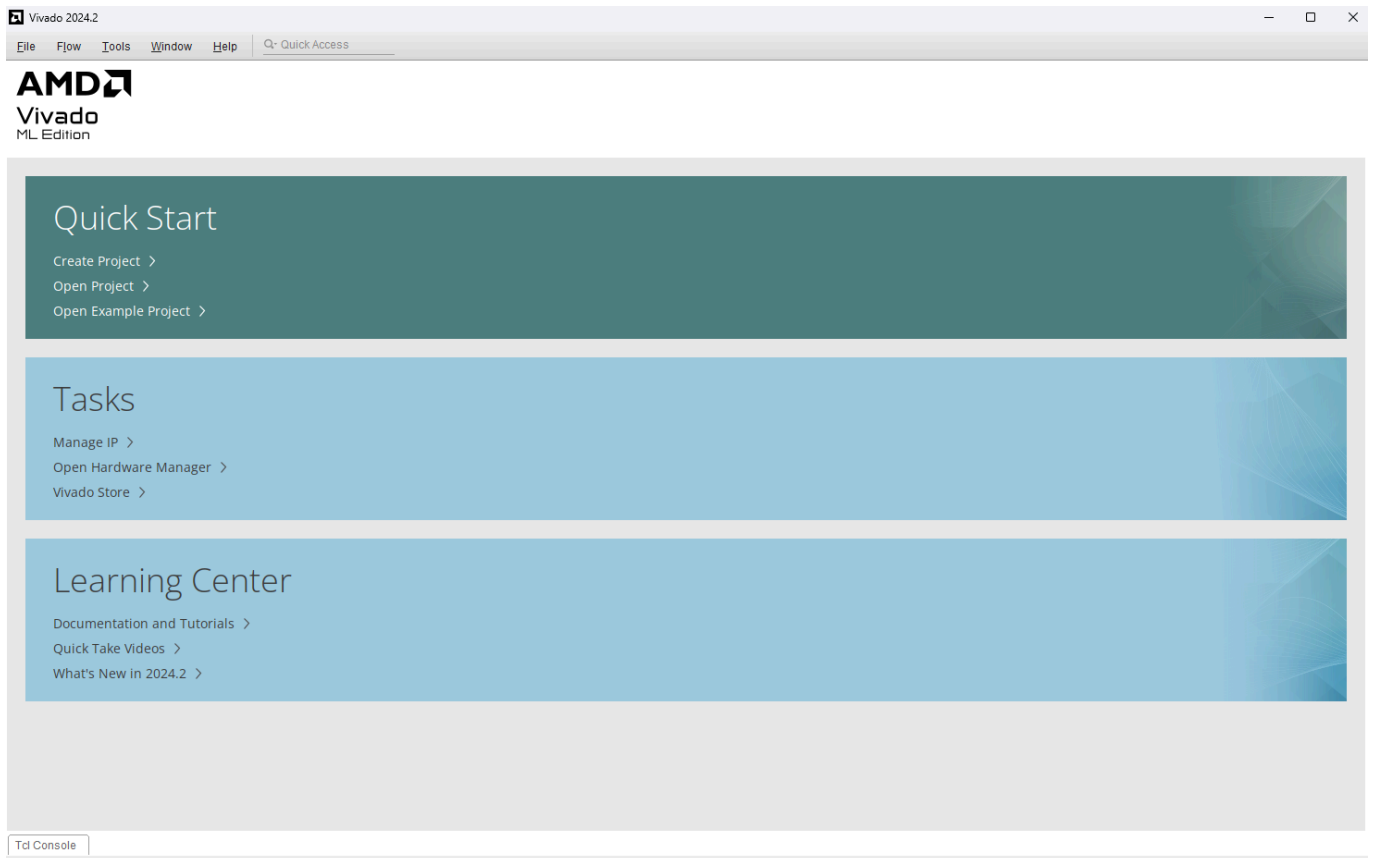


The screenshot shows a window titled "AMD Unified Installer for FPGAs & Adaptive SoCs 2024.2 - Accept License Agreements". The window has a dark header bar with the AMD logo on the right. Below the header, there is a section titled "Accept License Agreements" with a sub-header "Please read the following terms and conditions and indicate that you agree by checking the I Agree checkboxes." The main content area contains five sections, each for a different software component: "End User License Agreement for Vivado", "End User License Agreement for DocNav", "End User License Agreement for Model Composer", "End User License Agreement for PDM", and "End User License Agreement for Vitis Embedded". Each section contains a paragraph of text explaining the license terms and a checkbox labeled "I Agree", which is checked in all five sections. At the bottom of the window, there is a footer with copyright information: "Copyright © 1986-2022 Xilinx, Inc. All rights reserved." and "Copyright © 2022-2025 Advanced Micro Devices, Inc. All rights reserved." On the right side of the footer, there are three buttons: "< Back", "Next >", and "Cancel".

7. Si la instalación fue satisfactoria en el escritorio encontrará el siguiente icono. Haga doble click en el icono y espere, para verificar si se abre el software instalado.



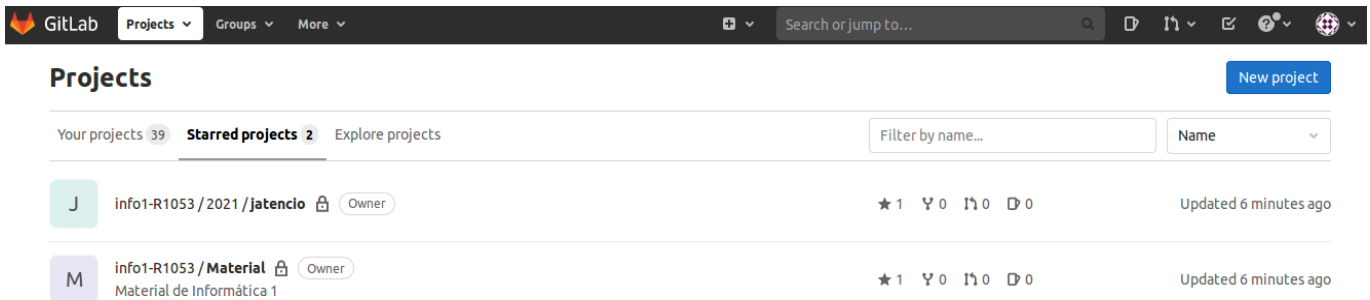
8. Finalmente debería ver una pantalla como la siguiente.



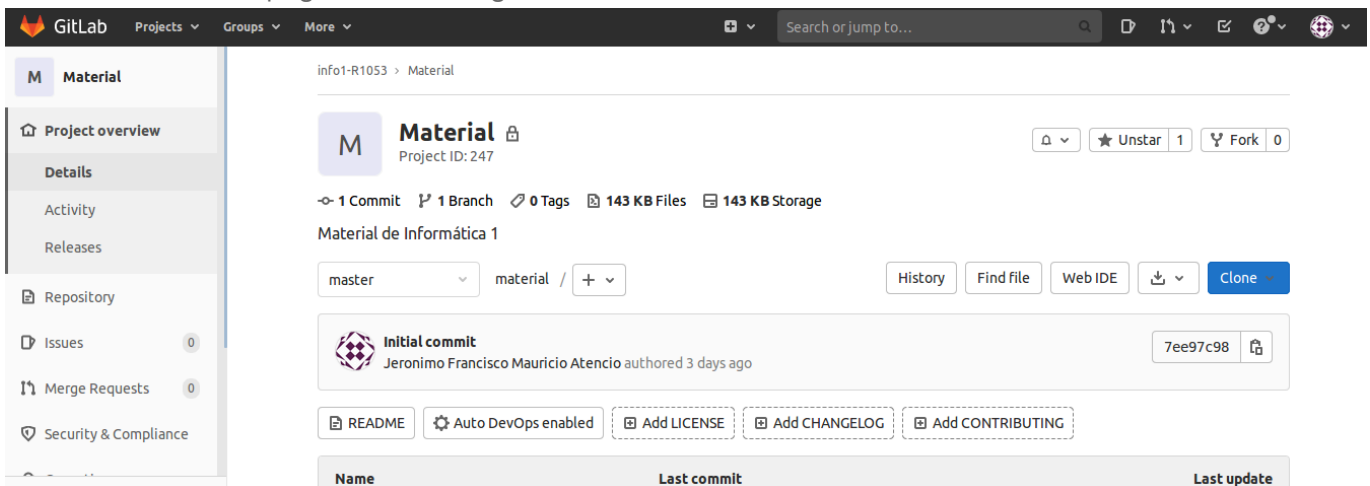
2. Accediendo al repositorio

1. Accedera a una página similar a esta donde están al menos dos repositorios
 - a. material: Es el repositorio para descargar el material de la materia
 - b. Un repositorio con su usuario SinAp: Este repositorio lo usarán para subir los trabajos prácticos.

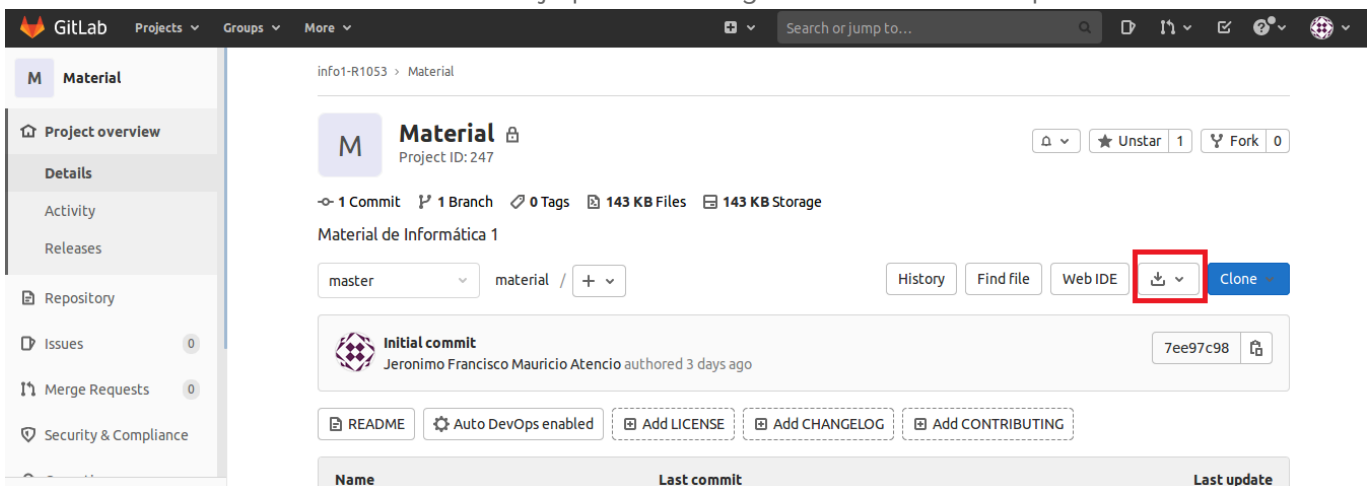
Seleccione el repositorio de material



2. Accedera a una página como la siguiente



3. Seleccionando el icono indicado en rojo podrá descargar el contenido del repositorio.



Configurando Git por primera vez

Antes de comenzar a usar el git deberá ejecutar los siguientes comandos para configurar su usuario e email. Reemplace lo indicado en azul por sus datos

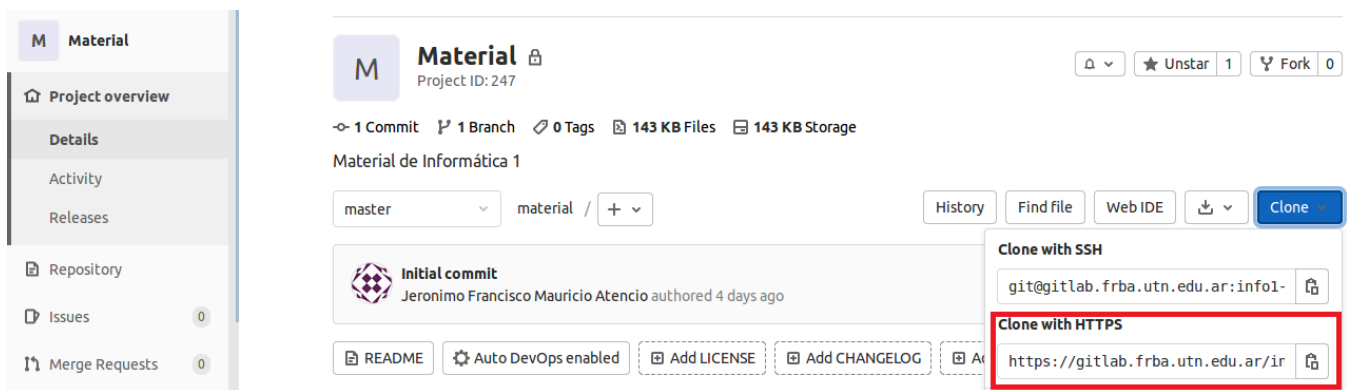
```
jerome@linuxVm:~$ git config --global user.name "usuario"
jerome@linuxVm:~$ git config --global user.email "email"
```

Utilizando el siguiente comando confirme que los datos se configuraron correctamente.

```
jerome@linuxVm:~$ git config --global --list
```

Clonando el repositorio de material.

1. Obtener la dirección del repositorio de material. Para ello entre en el repositorio a clonar, presione el botón **Clone** y copie la dirección marcada con el recuadro rojo.



2. La dirección obtenida en el punto anterior se coloca luego del comando git clone. Reemplazan lo indicado en azul por la dirección obtenida en el punto anterior.

```
jerome@linuxVm:~$ git clone direccionRepositorio
```

Luego de ejecutar el comando para realizar la clonación, se le solicitará el usuario y el password para acceder al repositorio. Si los datos son correctos se creará un directorio con el nombre del repositorio al cual puede acceder con el comando **cd**.

Por ejemplo para el repositorio de material quedaría

```
jerome@linuxVm:~/jatencio$ git clone
https://gitlab.frba.utn.edu.ar/td1-r3001/material.git
Cloning into 'material'...
Username for 'https://gitlab.frba.utn.edu.ar':
Password for 'https://jatencio@gitlab.frba.utn.edu.ar':
remote: Enumerating objects: 3, done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 3
Unpacking objects: 100% (3/3), done.
Checking connectivity... done.
```

Actualizando el la versión local del repositorio de material desde uno remoto (pull)

1. Ingresar a la carpeta del repositorio de material utilizando el comando **cd**.

```
jerome@linuxVm:~$ cd material  
jerome@linuxVm:~/material$
```

2. Realizamos un pull del repositorio remoto. (Lo que está en azul debe reemplazarlo por su nombre de usuario)

```
jerome@linuxVm:~/material$ git pull  
Username for 'https://gitlab.frba.utn.edu.ar': usuario  
Password for 'https://usuario@gitlab.frba.utn.edu.ar':  
remote: Enumerating objects: 5, done.  
remote: Counting objects: 100% (5/5), done.  
remote: Compressing objects: 100% (3/3), done.  
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0  
Unpacking objects: 100% (3/3), done.  
From https://gitlab.frba.utn.edu.ar/td1-r3001/material  
    c6d8a8a..97a0a6a  master      -> origin/master  
Updating c6d8a8a..97a0a6a  
Fast-forward  
  README.md | 5 ++--  
  1 file changed, 2 insertions(+), 3 deletions(-)
```

■ ■ ■

4. Algebra de boole. Minimización de funciones lógicas.

- Demuestre las siguientes igualdades: y
 - $x + yz = (x + y)(x + z)$
 - $(x + y)(x + \bar{y}) = x$
 - $xy + x\bar{y} = x$
- Dadas las siguientes funciones lógicas implemente el circuito más simple como suma de productos y como producto de sumas.
 - $f(c, b, a) = \sum(m_0; m_1; m_4; m_7)$
 - $f(c, b, a) = \sum(m_0; m_2; m_5; m_7)$
 - $f(c, b, a) = \prod(m_1; m_2; m_5; m_7)$
 - $f(d, c, b, a) = \sum(m_0; m_1; m_2; m_3; m_7; m_{12}; m_{13}; m_{15})$
 - $f(d, c, b, a) = \prod(m_0; m_1; m_5; m_7; m_8; m_{10}; m_{14}; m_{15})$
- Repita el punto anterior usando sólo compuertas NAND y NOR
- Implemente un circuito decodificador según la siguiente tabla de la verdad.

W(1)	W(0)	Y(3)	Y(2)	Y(1)	Y(0)
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

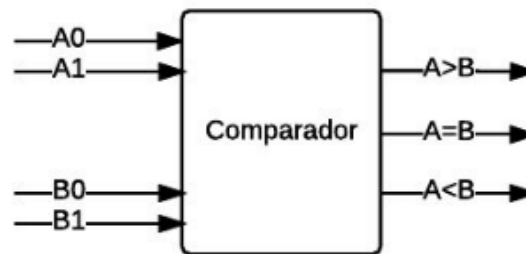
- Al circuito anterior, agregue dos entradas una activa nivel bajo E(0) y otra activa nivel alto E(1) que permita poner en uno todas las salidas.

E(1)	E(0)	W(1)	W(0)	Y(3)	Y(2)	Y(1)	Y(0)
1	0	0	0	1	1	1	0
1	0	0	1	1	1	0	1
1	0	1	0	1	0	1	1
1	0	1	1	0	1	1	1
0	0	-	-	1	1	1	1
0	1	-	-	1	1	1	1
1	1	-	-	1	1	1	1

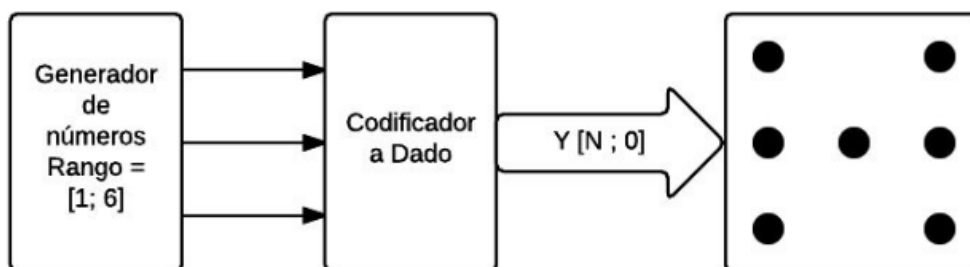
6. Sintetizar un circuito codificador con la siguiente tabla de la verdad.

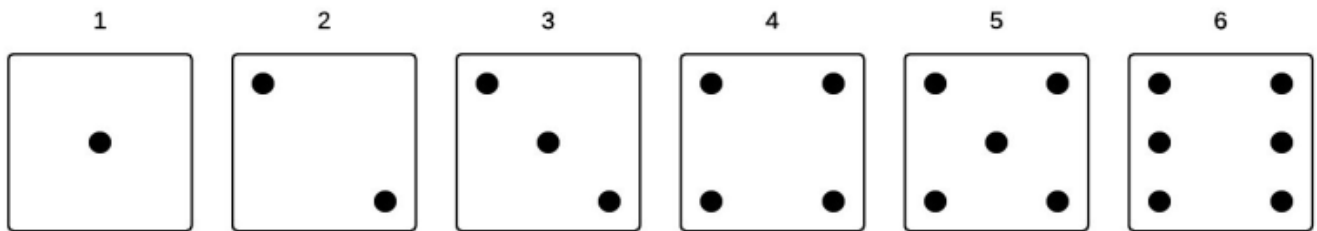
W(3)	W(2)	W(1)	W(0)	Y(1)	Y(0)
1	1	1	0	0	0
1	1	0	1	0	1
1	0	1	1	1	0
0	1	1	1	1	1
El resto de las combinaciones				0	0

7. Implemente un comparador de magnitudes de 2 bits. Siendo A1 y B1 los bits más significativos de las palabras A y B.

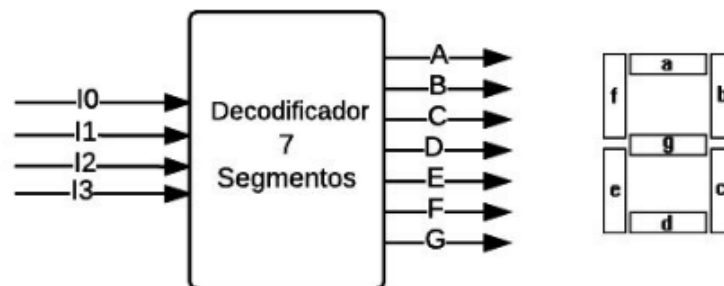


- Implemente la tabla de la verdad
 - Obtenga las funciones lógicas mínimas
 - Dibuje el circuito resultante.
 - Implemente utilizando multiplexores con 4 entradas de control
 - Implemente utilizando multiplexores con 3 entradas de control. Escriba la tabla de la verdad reducida.
8. Se dispone de un generador de números pseudoaleatorios de tres bits cuyo rango es [1; 6]. Realice una función lógica que convierta estos números en la representación de las caras de un dado. Cada punto del dado se prende con un '1'.





- Determine la cantidad de salidas del codificador a dado.
 - Obtenga las funciones lógicas del codificador a dado.
 - Dibuje el circuito resultante.
 - Dibuje el diagrama temporal para la secuencia de entrada 1; 2; 3; 4; 5; 6.
 - ¿Qué LEDs del dado se encenderán si como entrada se coloca alguna de estas combinaciones B"000" ó B"111"?
9. Implemente un decodificador de BCD a 7 segmentos. El display de cátodo común, es decir, cada segmento se prende con un '1' lógico.



- Obtenga las funciones lógicas mínimas.
- Dibuje el circuito resultante.
- Implemente usando un decodificador 4 a 16
- Implemente utilizando un multiplexor con 4 entradas de control.
- Implemente utilizando un multiplexor con 3 entradas de control. Escriba la tabla de la verdad reducida.



5. Aritmética

1. Implemente un circuito que realice la multiplicación de dos números de dos bits signados en CA2.
 - a. Definir la cantidad de bits del resultado.
 - b. Realizar tabla de verdad.
 - c. Obtenga las funciones lógicas mínimas.
2. Implemente un comparador de números signados en CA2 de N bits utilizando como base un comparador de magnitud.
3. Implemente un circuito que realice **desplazamiento a la izquierda** de un número 8 bits sin signo (**magnitud**) y complete con ceros la parte menos significativa (Least Significant Bit: LSB).

Donde:

- A: Es el número a desplazar. (magnitud)
- D: Es la cantidad de bits a desplazar. (magnitud)
- R: Es el número desplazado.

Ejemplos:

A	D	R
0001_0101	000	0001_0101
1001_0101	001	0010_1010
0000_0001	111	1000_0000

4. Implemente un circuito que realice **desplazamiento a la derecha** de un número 8 bits **signado en CA2**.

Donde:

- A: Es el número a desplazar. (signado CA2)
- D: Es la cantidad de bits a desplazar. (magnitud)
- R: Es el número desplazado.

Ejemplos:

A	D	R
0001_0101	000	0001_0101
1001_0101	001	1100_1010
0100_0001	110	0000_0001

5. Implemente un circuito que realice **desplazamiento a la izquierda** de un número 8 bits **signado en CA2**.

Donde:

- A: Es el número a desplazar. (signado CA2)
- D: Es la cantidad de bits a desplazar. (magnitud)
- R: Es el número desplazado.

Ejemplos:

A	D	R
0001_0101	000	0001_0101
1001_0101	001	1100_1010
0100_0001	110	0000_0001

6. Implemente un circuito que reciba un número signado en CA2 de 16 bits y sature el mismo a 8 bits.
7. Implemente un circuito que realice la suma de dos números signados en CA2 de N bits con resultado de N bits. Si la operación produce overflow sature el resultado. Indique con una salida que el resultado está saturado. Realice un diseño a partir de celdas Full Adder.
8. Implemente un circuito que realice la resta de dos números signados en CA2 de N bits con resultado de N bits. Si la operación produce overflow sature el resultado. Indique con una salida que el resultado está saturado. Realice un diseño a partir de celdas Full Adder.
9. Implemente un circuito a nivel de compuertas que realice el decremento de 2 a un número signado en CA2 de N bits. El circuito deberá indicar si hubo overflow al realizar la operación. No está permitido el uso de sumadores, ni restadores. Optimice la cantidad de compuertas usadas.
10. Implemente un circuito a nivel de compuertas que realice el incremento de 2 a un número signado en CA2 de N bits. El circuito deberá indicar si hubo overflow al realizar la operación. No está permitido el uso de sumadores, ni restadores. Optimice la cantidad de compuertas usadas.
11. Implemente un circuito que posea 5 entradas y cuente la cantidad de unos que hay en ellas.
 - a. Determine la cantidad de bits de salida del circuito.
 - b. Implemente utilizando compuertas. Puede dividir el diseño en bloques.
 - c. Implemente utilizando sumadores de N bits
12. Implemente un circuito que permita calcular el promedio de 4 números binarios signados en CA2 de 4 bits cada uno.
 - a. Determine la cantidad de bits de salida del circuito. (Trunque la parte decimal del número obtenido.)
 - b. Implemente utilizando sumadores.
13. Implemente un circuito combinacional que realice la multiplicación del número ingresado por la constante $3/2$ (tres medios). La entrada es una magnitud con una N bits
 - a. Determine la cantidad de bits de salida del circuito. (Trunque la parte decimal del número obtenido.)
 - b. Implemente utilizando sumadores y la lógica combinacional necesaria.



6. Circuitos combinacionales.

Ejemplos

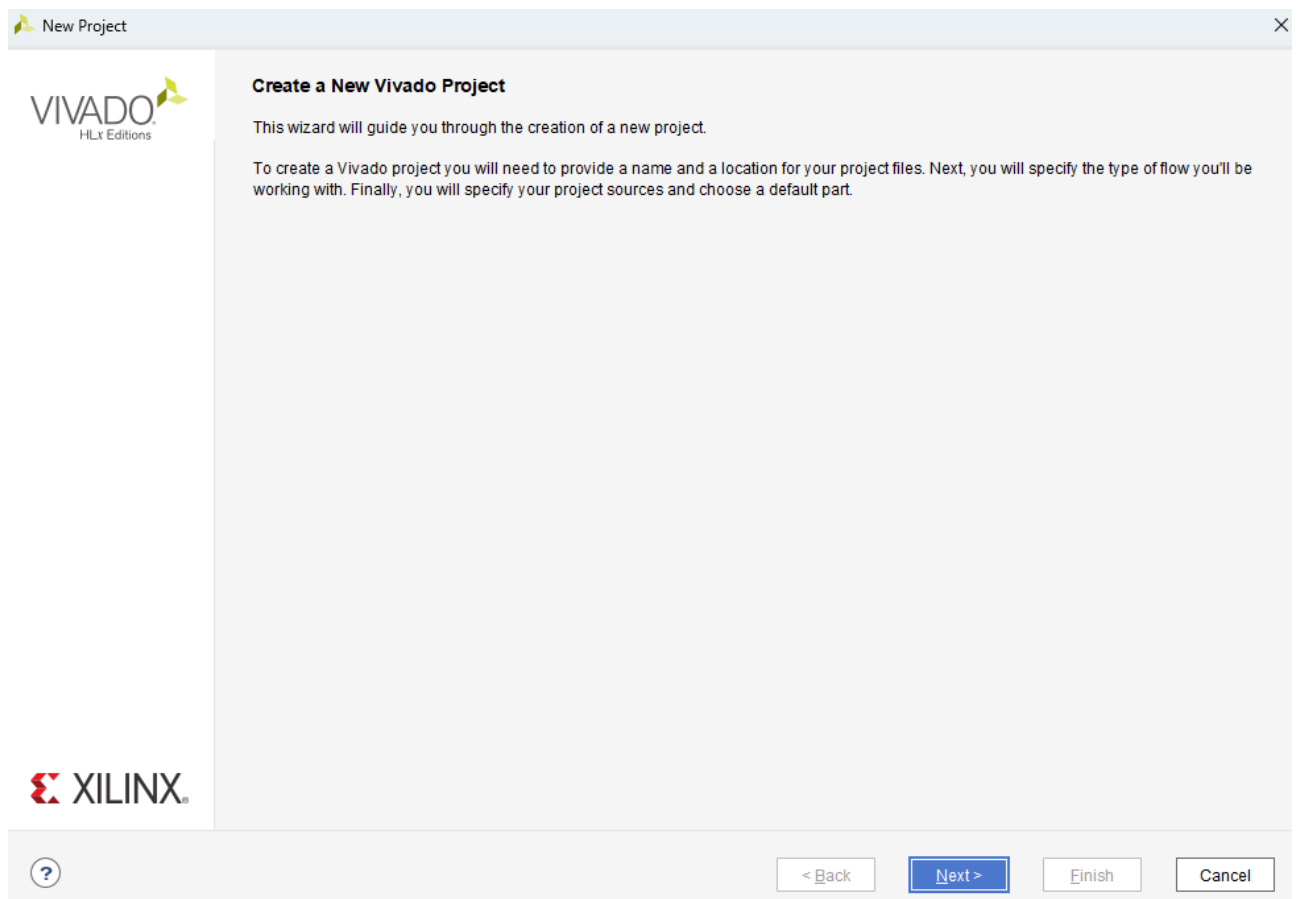
1. En este ejemplo se implementara una compuerta and de dos entradas en vivado junto con su simulación. A continuación se presenta la entidad de la compuerta and

```
entity myAnd2 is  
  Port ( a, b : in std_logic;  
         y : out std_logic);  
end myAnd2;
```

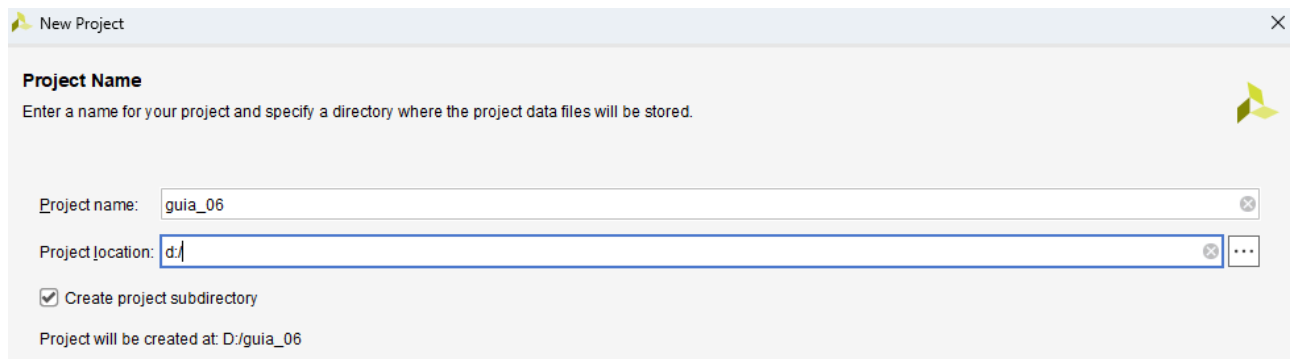
- a. Abra el vivado utilizando el siguiente icono.



- b. Cree un nuevo proyecto **File → New** Aparecerá la siguiente ventana en la cual deberá presionar el boton **Next**



- c. Coloque el nombre del proyecto. En este caso guia_06, en **Project location** coloque el directorio donde se alojara la carpeta del proyecto y deje seleccionado el check **Create project subdirectory**. Luego presione el botón **Next**



New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

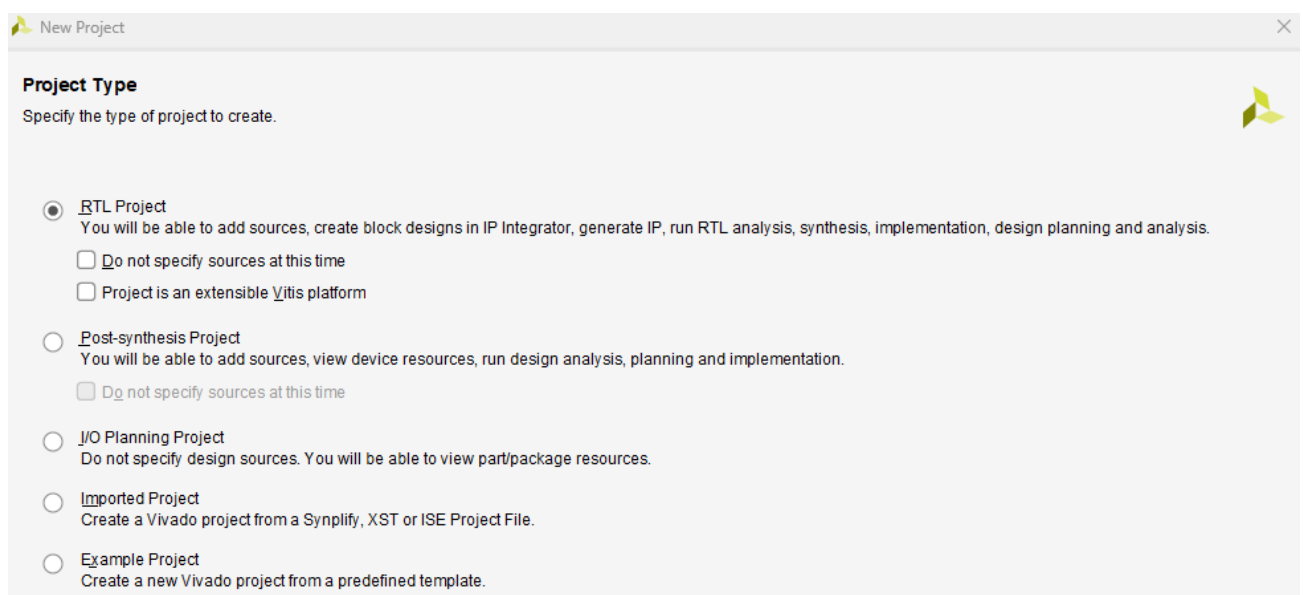
Project name:

Project location:

☒ Create project subdirectory

Project will be created at: D:/guia_06

- d. Seleccione RTL Project en la siguiente ventana. Presione el botón **Next**



New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☐ Do not specify sources at this time
☐ Project is an extensible Vitis platform

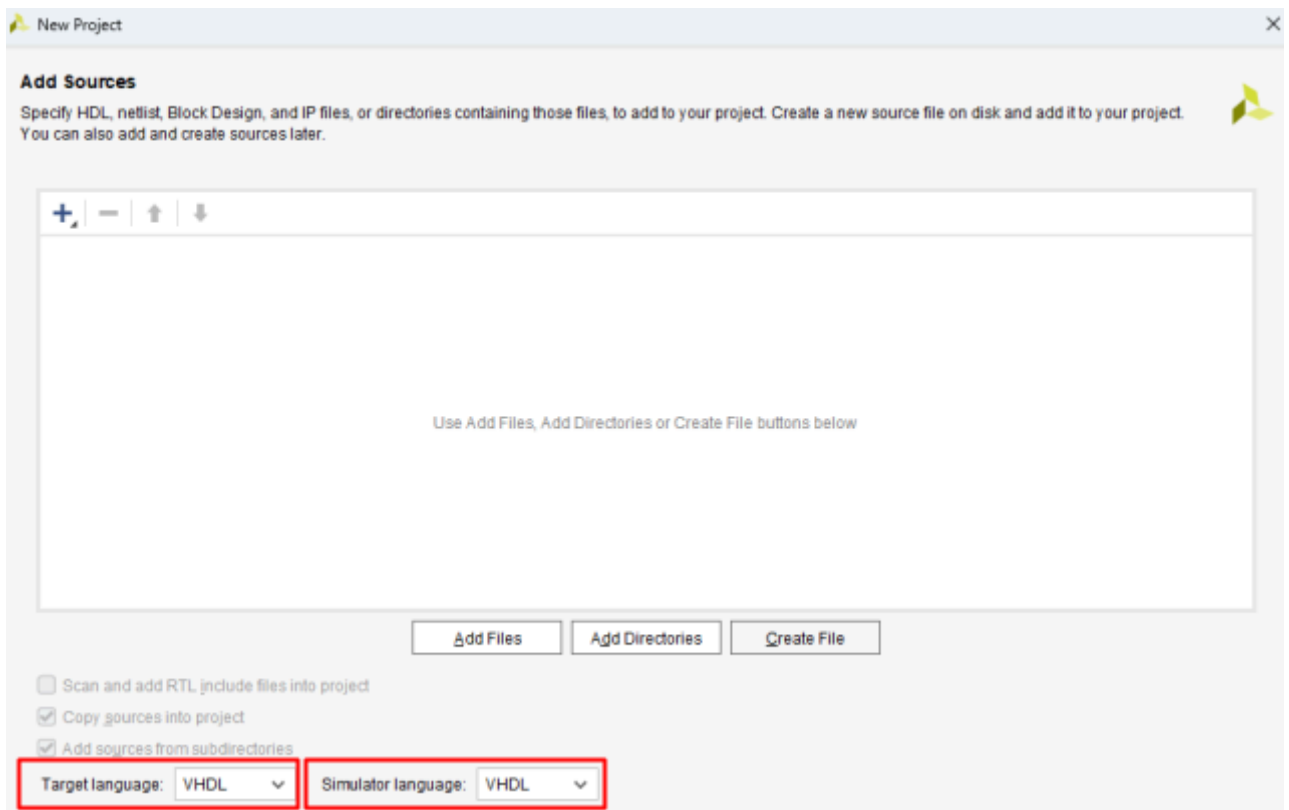
☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

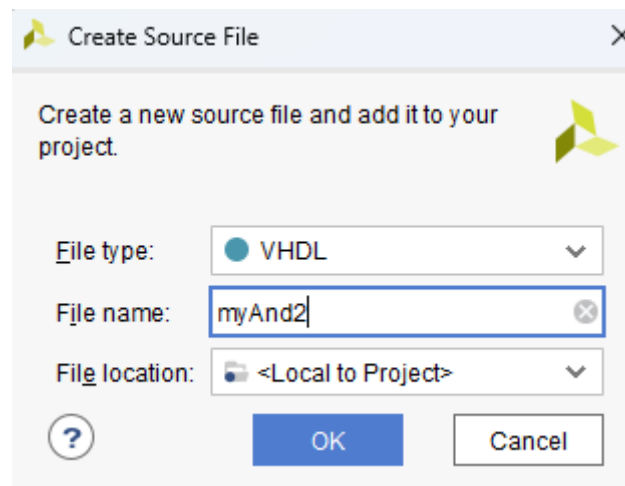
☐ **Imported Project**
Create a Vivado project from a Synplify, XST or ISE Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

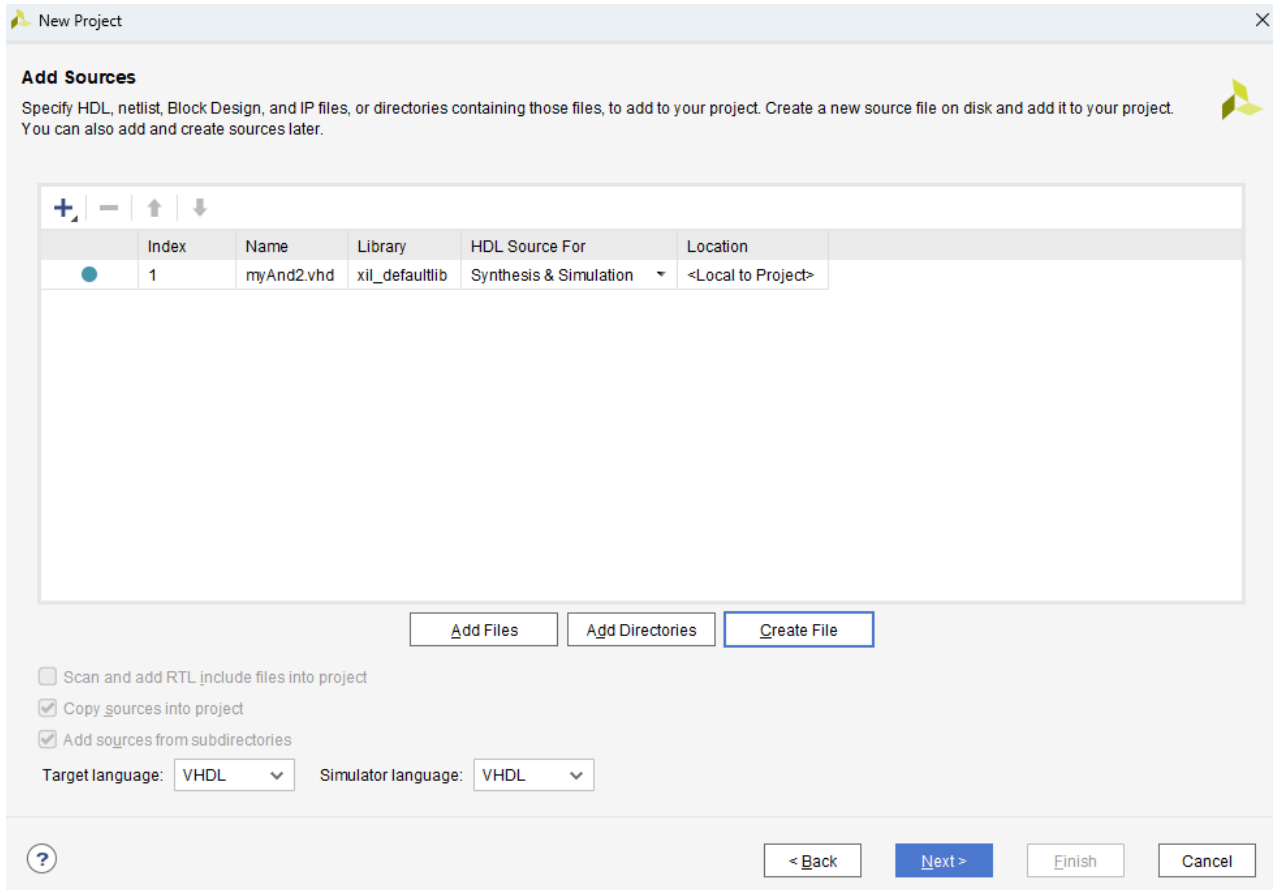
- e. Verifique en la siguiente ventana que tanto el **Target language** como el **Simulator language** estén seleccionados como VHDL, ambas opciones las encontrará recuadradas en rojo en la siguiente imagen. Presione el botón **Create File**, para crear un nuevo archivo.



- f. Verifique que el **File Type** esté en VHDL, y coloque el nombre del archivo a crear que deberá coincidir con el nombre de la entidad. Presione el botón **OK**



- g. Si necesita crear más archivos repita los pasos anteriores, en caso contrario presione el botón **next**.



Add Sources

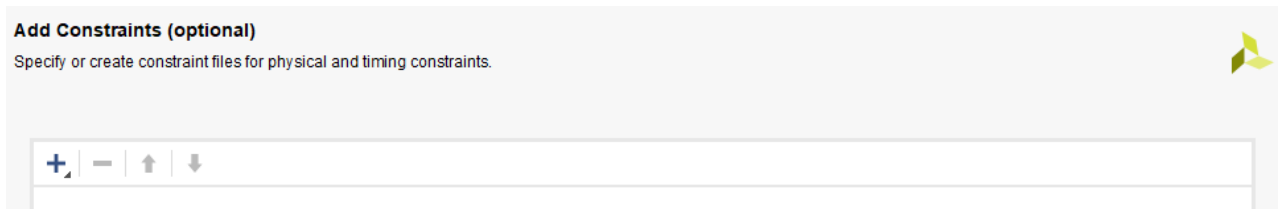
Specify HDL, netlist, Block Design, and IP files, or directories containing those files, to add to your project. Create a new source file on disk and add it to your project. You can also add and create sources later.

	Index	Name	Library	HDL Source For	Location
	1	myAnd2.vhd	xil_defaultlib	Synthesis & Simulation	<Local to Project>

☐ Scan and add RTL include files into project
☒ Copy sources into project
☒ Add sources from subdirectories

Target language: Simulator language:

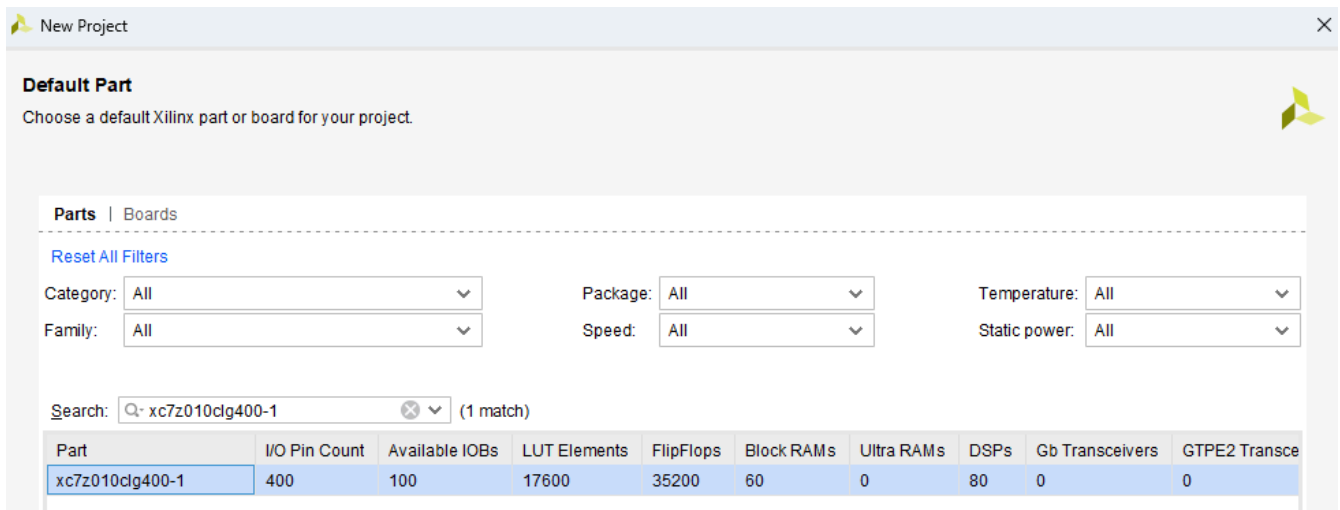
- h. Por el momento no agregaremos archivo de constraints así que presione el botón **Next**



Add Constraints (optional)

Specify or create constraint files for physical and timing constraints.

- i. Seleccione el siguiente dispositivo xc7z010clg400-1 y presione **Next**



Default Part

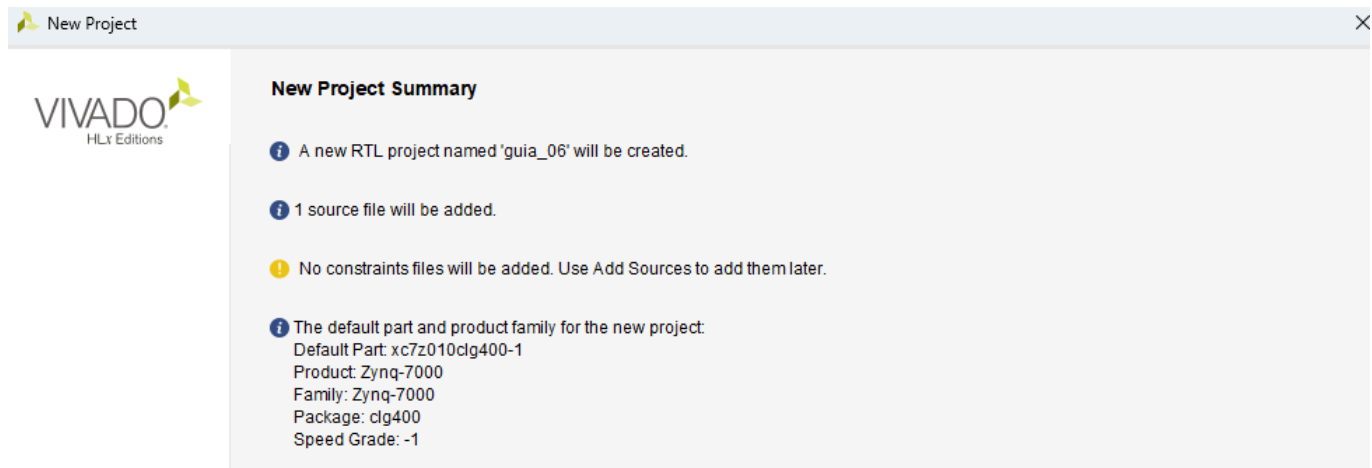
Choose a default Xilinx part or board for your project.

Category: Package: Temperature:
 Family: Speed: Static power:

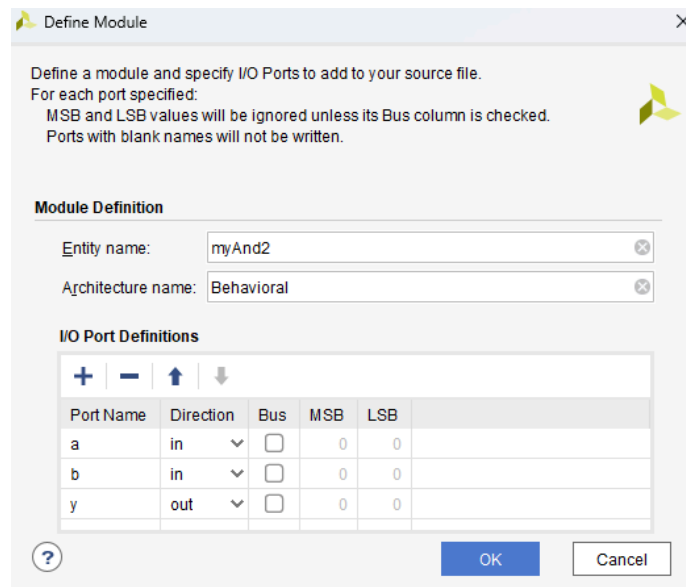
Search: (1 match)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	Gb Transceivers	GTPE2 Transceivers
xc7z010clg400-1	400	100	17600	35200	60	0	80	0	0

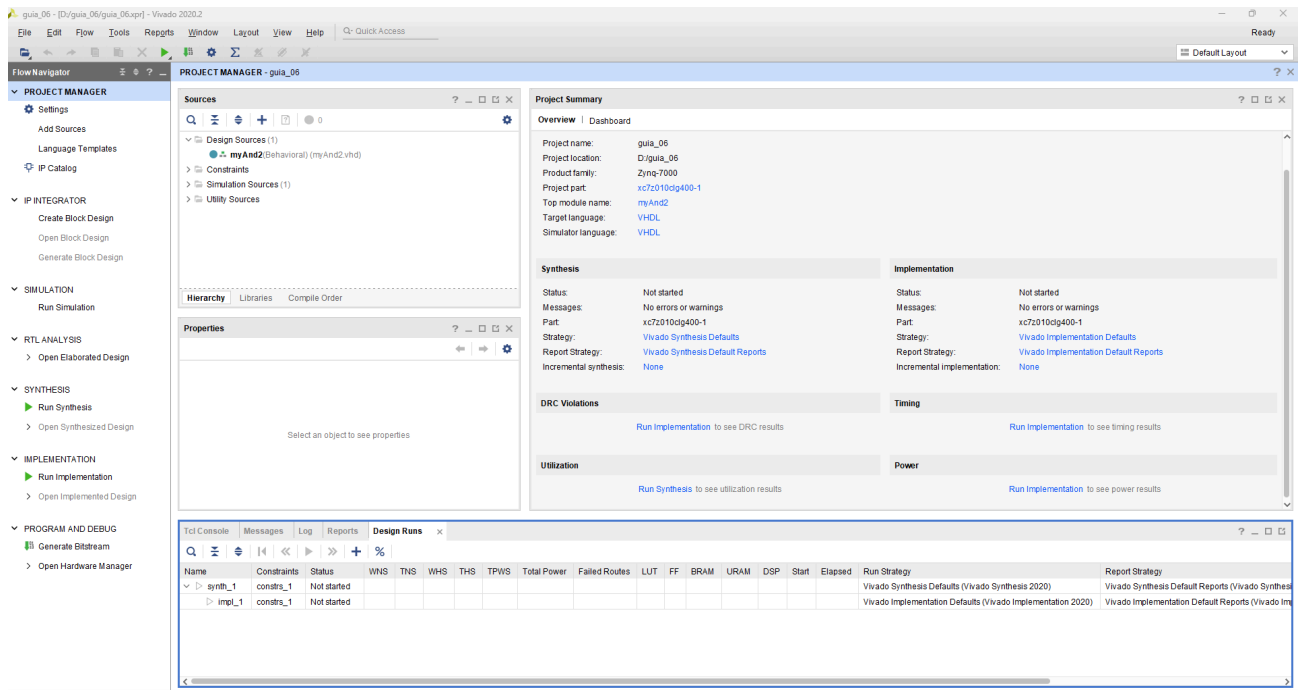
j. Confirme el resumen del proyecto y presione el botón **Finish**



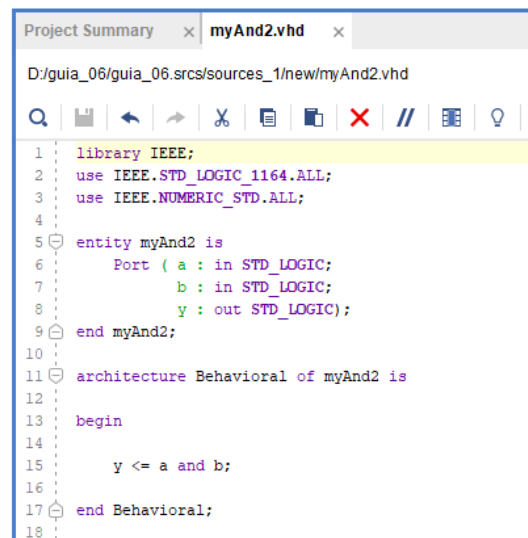
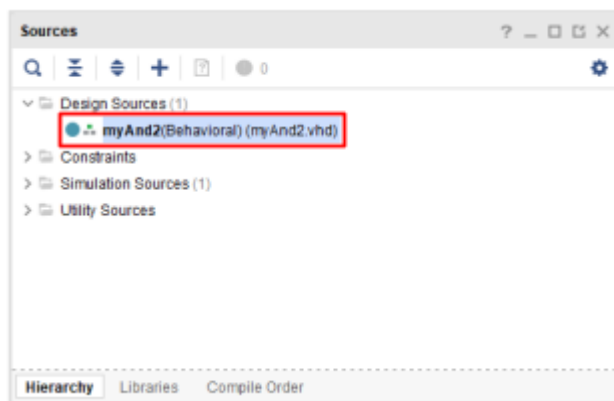
k. Al Iniciar el proyecto se abrirá la siguiente ventana donde completará con los ports de la entidad a implementar



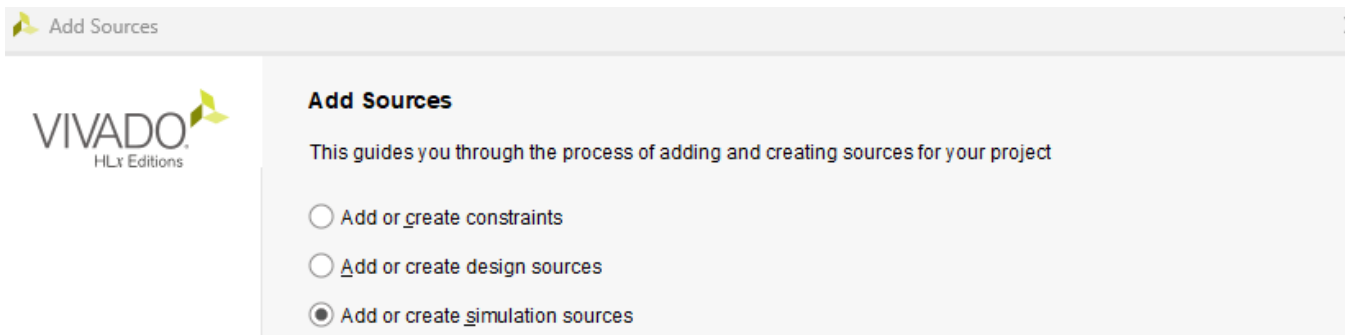
l. El proyecto se verá como en la siguiente imagen.



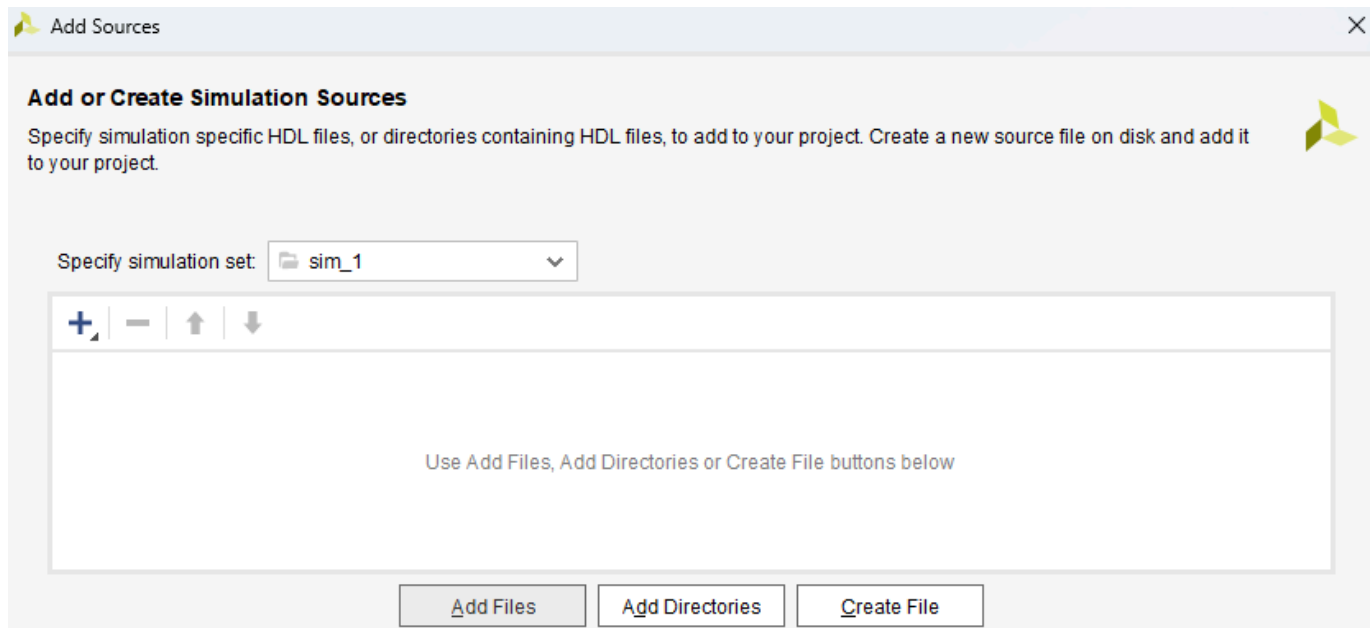
m. En la ventana de sources haga doble clic sobre el archivo myAnd2. En la ventana que se abre coloque la descripción correspondiente a una and de dos entradas.



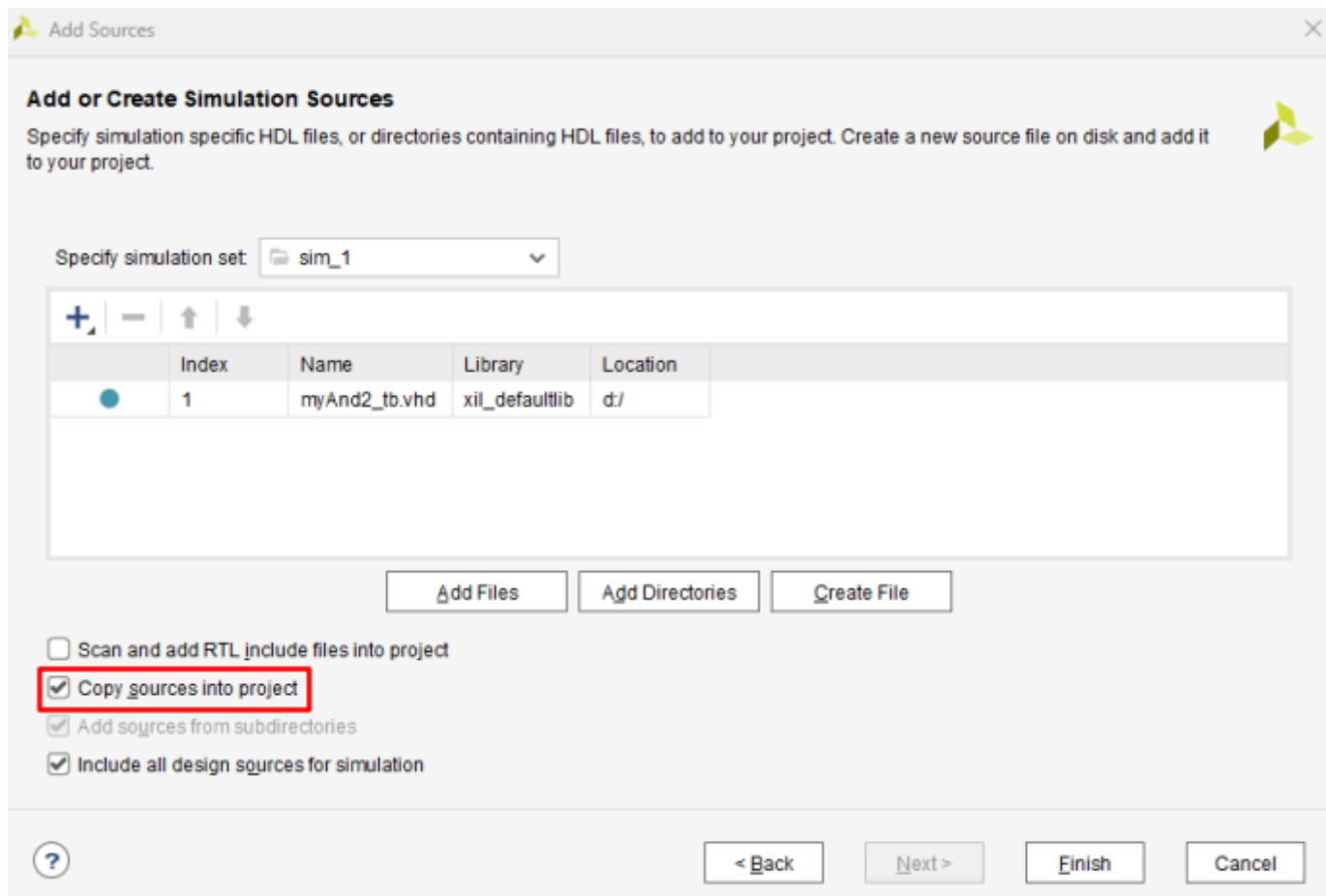
n. Agregue el archivo de simulación. En la ventana sources presione el botón +, en la ventana que se abre seleccione la opción **Add or create simulation sources** y presione el botón Next



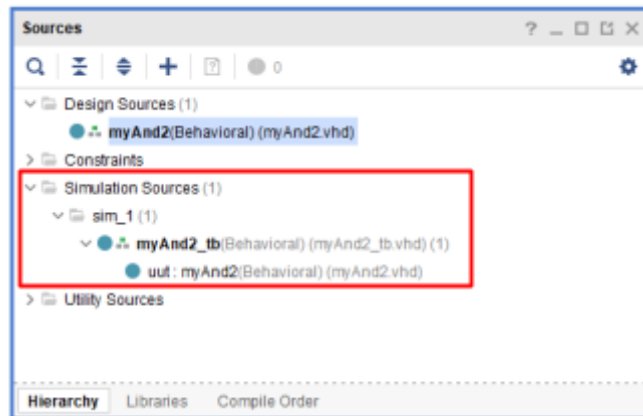
- o. En esta ventana presione el botón *Add files* y seleccione el archivo de simulación a agregar. El mismo lo encontrará en el repositorio de la materia con el y el nombre será *myAnd2_tb.vhd*



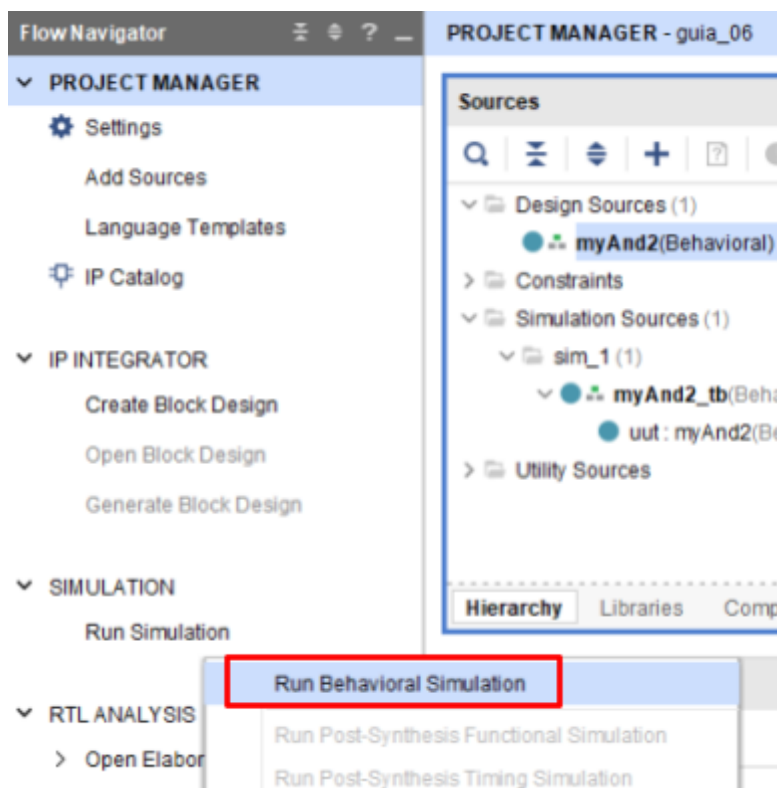
- p. Una vez agregado el proyecto



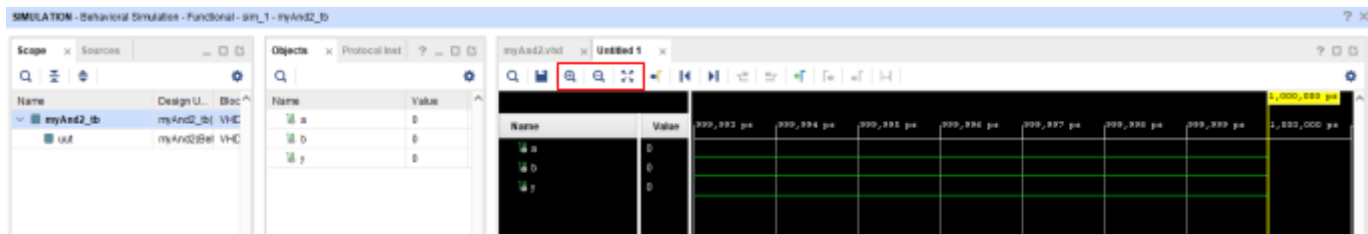
- q. Si el archivo de simulación se agregó correctamente en la ventana de **Sources** podrá observar lo indicado en el recuadro rojo de la siguiente imagen



- r. Agregar otro archivo al proyecto.

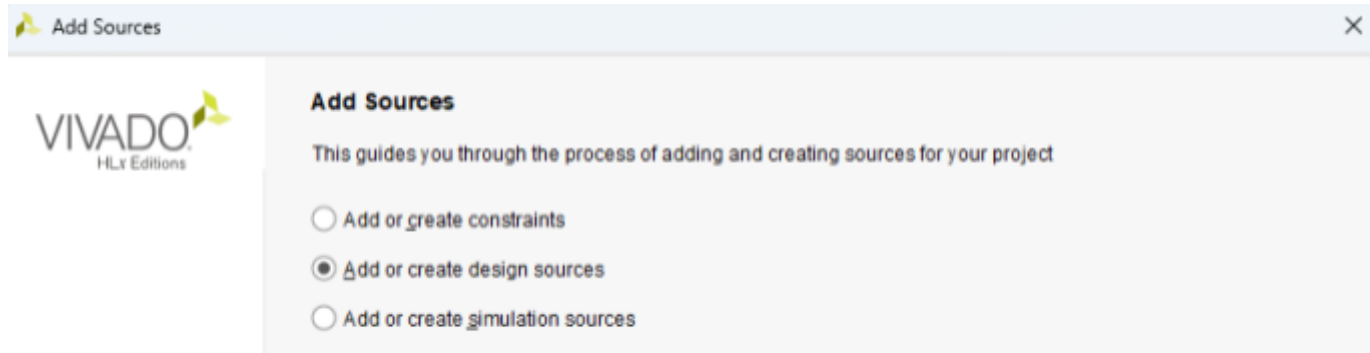


- s. Finalmente podrá observar el resultado de la simulación. Con los botones marcados en rojo podrá ajustar la visualización de la forma de onda. Y con la cruz en la esquina superior derecha sobre la barra celeste podrá cerrar la simulación.

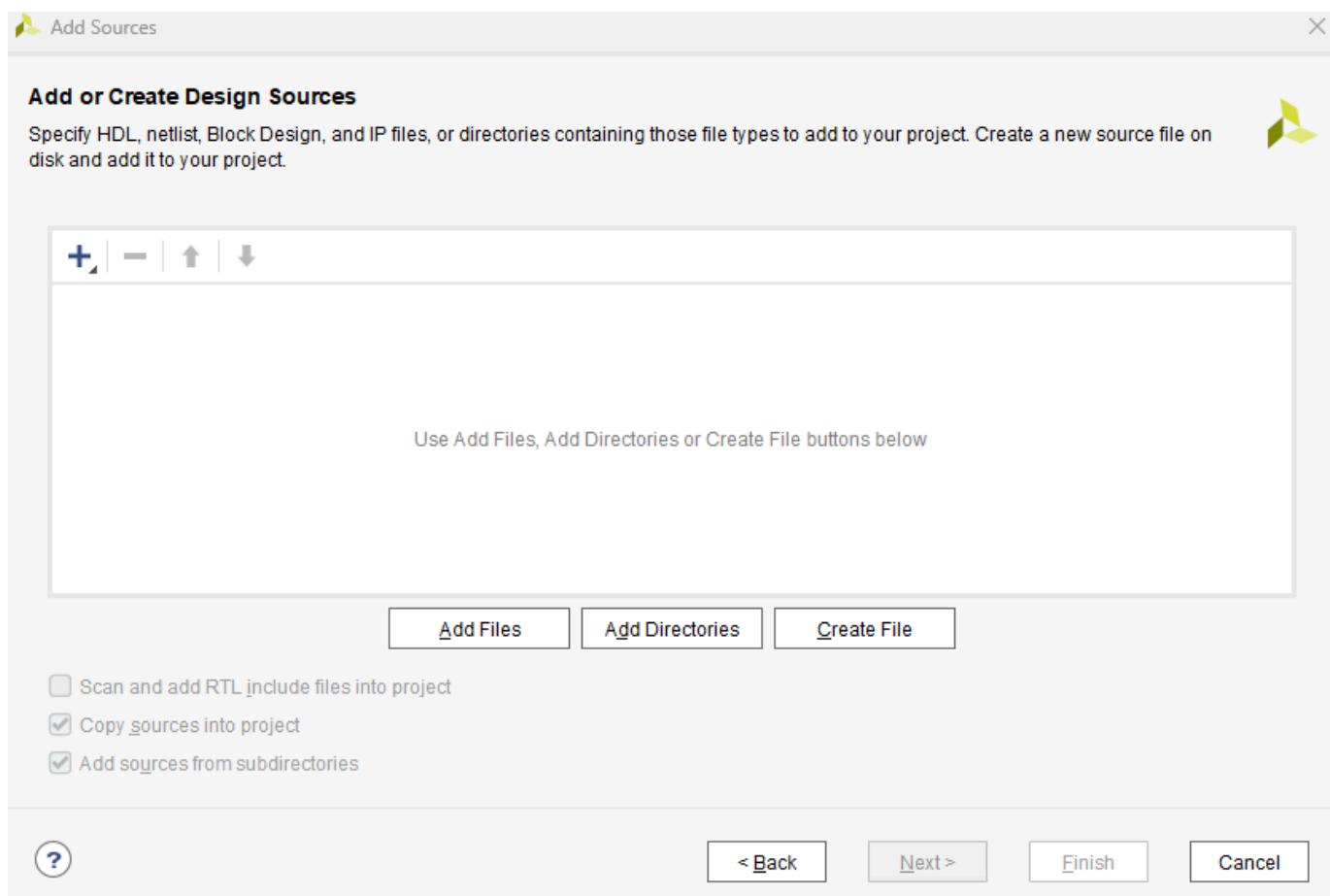


Agregar un nuevo archivo de diseño

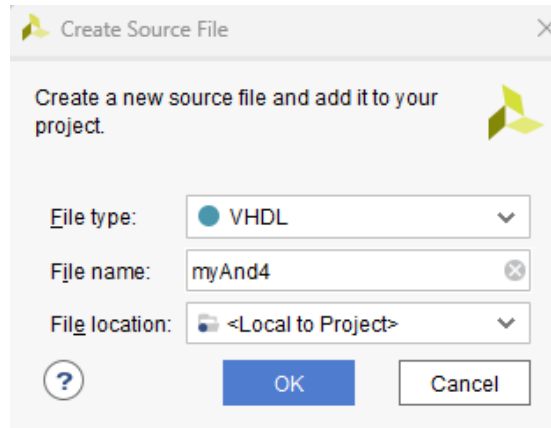
- a. Agregue el archivo de diseño. En la ventana *sources* presione el botón **+**, en la ventana que se abre seleccione la opción **Add or create design sources** y presione el botón **Next**



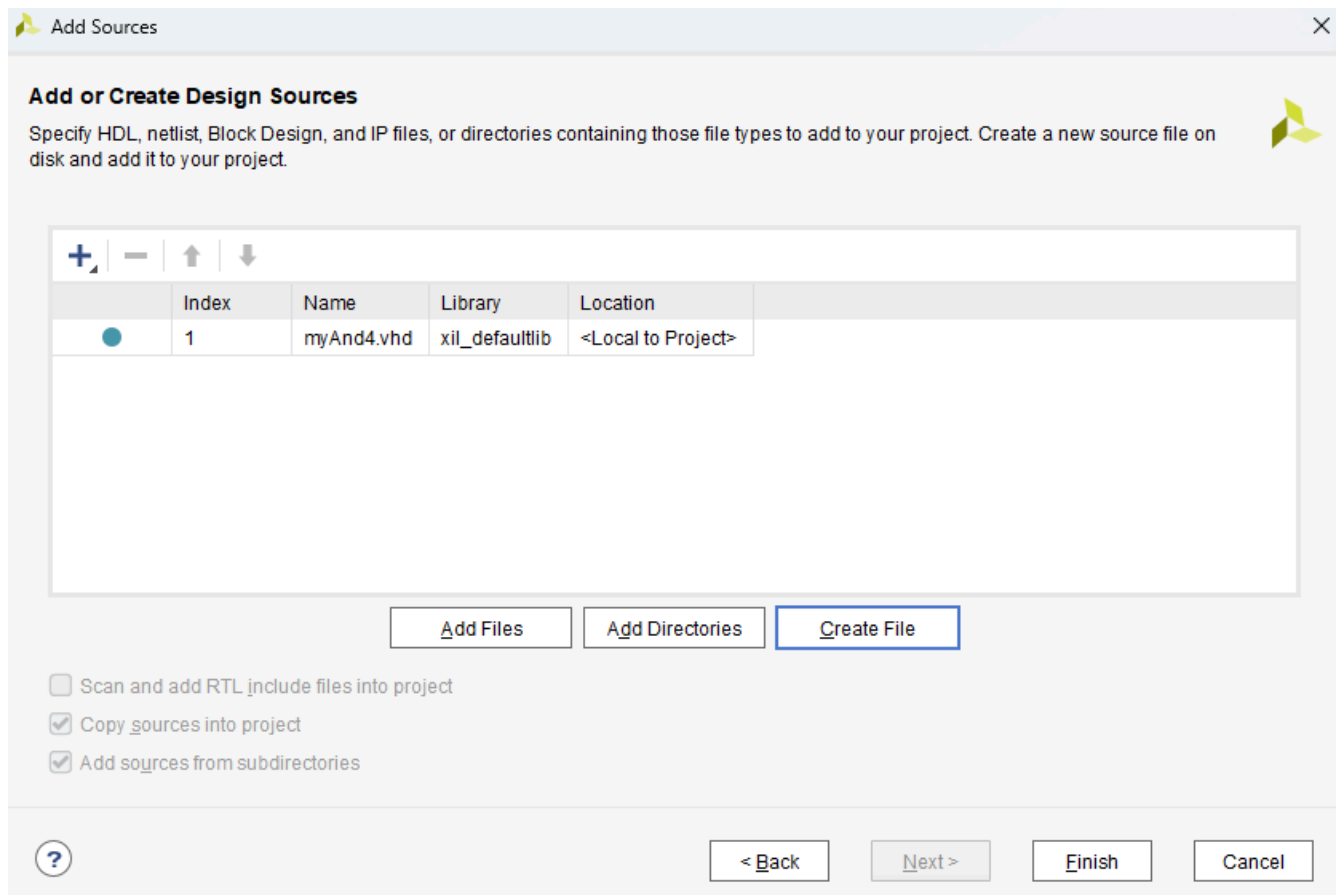
- b. En la siguiente ventana presione el botón **Create File**



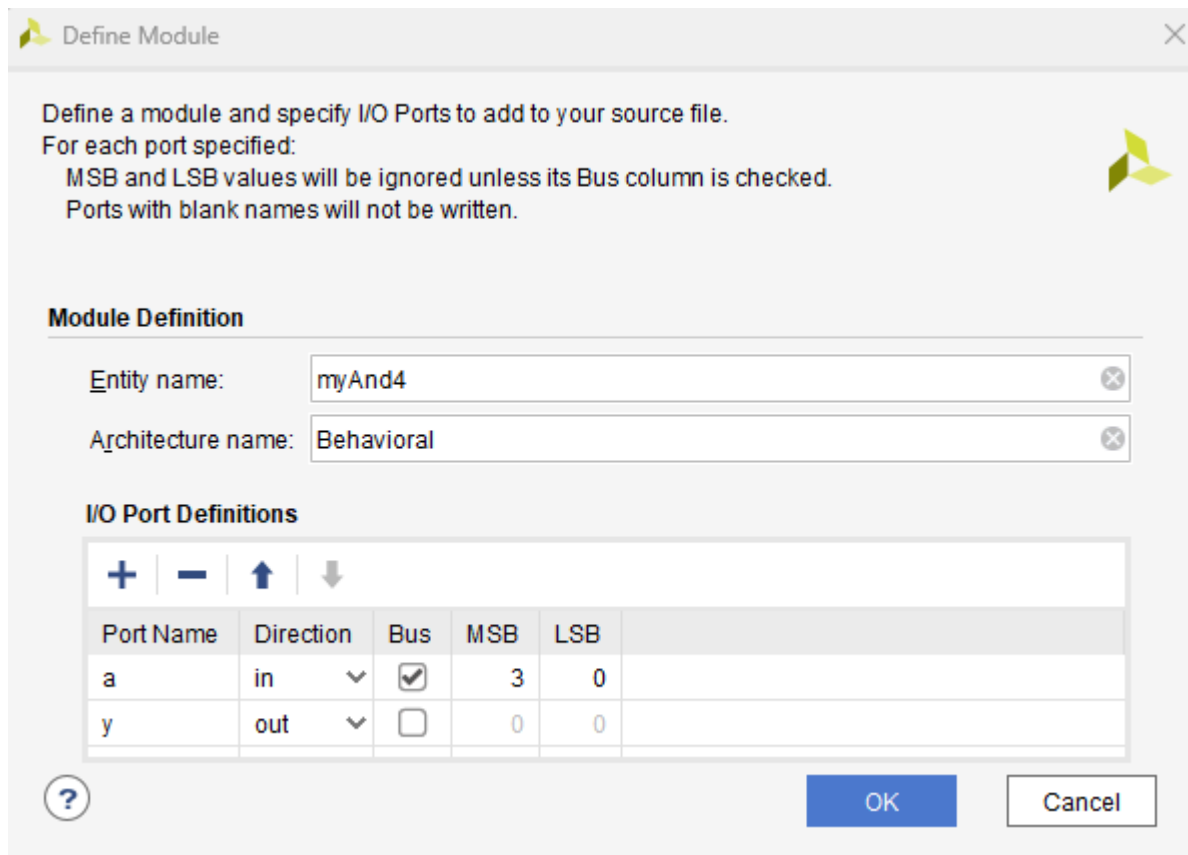
- c. Coloque el nombre del archivo a crear y verifique que el **File type** esté seleccionado VHDL



- d. Una vez creado el nuevo archivo lo podrá ver reflejado en la siguiente ventana. Finalmente presione **Finish**.



- e. Complete los ports en la siguiente ventana. Como el port a de entrada es un **std_logic_vector (3 downto 0)** Indicamos que el port es un bus y colocamos el rango del mismo. Al finalizar de cargar todos los ports presione el botón **OK**



Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Entity name:

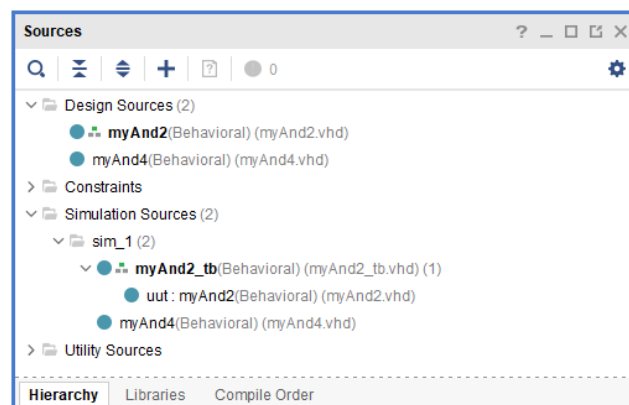
Architecture name:

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
a	in	☒	3	0
y	out	☐	0	0

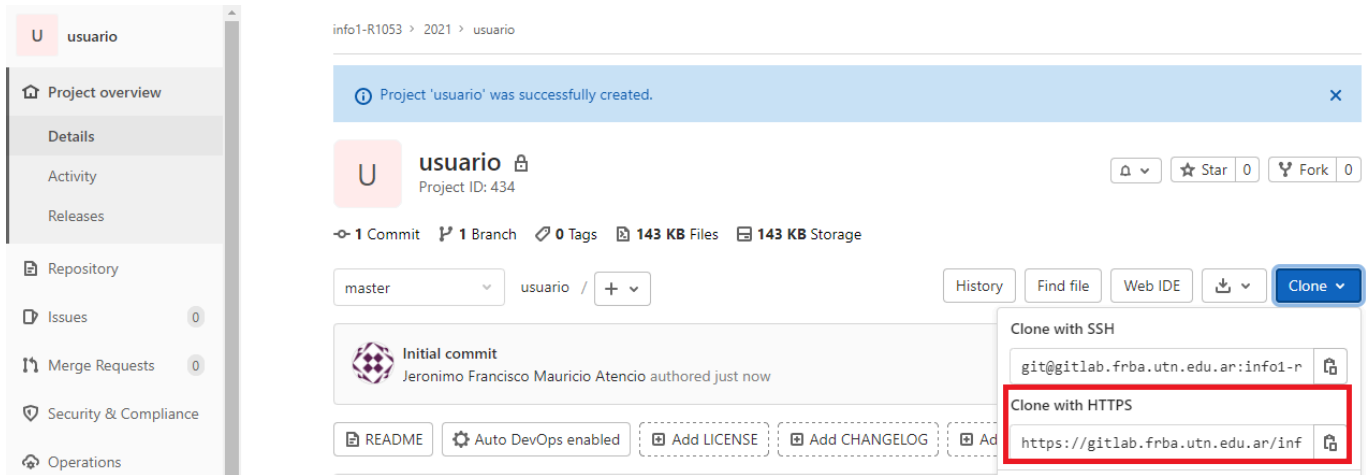
OK Cancel

- f. Si se agregó correctamente el nuevo archivo lo verán reflejado en la ventana de sources.



Clonando su repositorio personal.

1. Obtener la dirección del repositorio de su repositorio personal. Para ello entre en el repositorio a clonar, presione el botón **Clone** y copie la dirección marcada con el recuadro rojo.



2. La dirección obtenida en el punto anterior se coloca luego del comando **git clone**. Reemplazan lo indicado en azul por la dirección obtenida en el punto anterior.

```
jerome@linuxVm:~$ git clone direccionRepositorio
```

Luego de ejecutar el comando para realizar la clonación, se le solicitará el usuario y el password para acceder al repositorio. Si los datos son correctos se creará un directorio con el nombre del repositorio al cual puede acceder con el comando **cd**.

Por ejemplo para el repositorio de material quedaría

```
jerome@linuxVm:~/jatencio$ git clone
https://gitlab.frba.utn.edu.ar/td1-r3001/2025/usuario.git
Cloning into 'usuario'...
Username for 'https://gitlab.frba.utn.edu.ar':
Password for 'https://usuario@gitlab.frba.utn.edu.ar':
remote: Enumerating objects: 3, done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 3
Unpacking objects: 100% (3/3), done.
Checking connectivity... done.
```

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre `guia_06` e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. `xc7z010clg400-1`

1. Implemente una compuerta AND de cuatro entradas.

```
entity myAnd4 is
Port ( a : in std_logic_vector (3 downto 0);
      y : out std_logic);
end myAnd4;
```

2. Implemente las siguientes funciones lógicas utilizando los operadores lógicos

- $s = a \text{ xor } b \text{ xor } ci$
- $co = (A \text{ and } B) \text{ or } (B \text{ and } CI) \text{ or } (A \text{ and } CI)$

entity fa is

```
Port ( a, b, ci : in std_logic;  
      s, co : out std_logic);
```

end fa;

3. Implemente un multiplexor de 4 vías de datos.

entity myMux4 is

```
Port ( a : in std_logic_vector (3 downto 0);  
      c : in std_logic_vector (1 downto 0);  
      y : out std_logic);
```

end myMux4;

4. Implemente un decodificador de 3a8

entity deco3a8 is

```
Port ( w : in std_logic_vector (2 downto 0);  
      y : out std_logic_vector (7 downto 0));
```

end deco3a8;

5. Implemente un decodificador de 3a8 con enable

entity deco3a8Ena is

```
Port ( w : in std_logic_vector (2 downto 0);  
      e : in std_logic;  
      y : out std_logic_vector (7 downto 0));
```

end deco3a8Ena;

6. Implemente un codificador de 8a3 con prioridad

entity coder8a3 is

```
Port ( w : in std_logic_vector (7 downto 0);  
      y : out std_logic_vector (2 downto 0));
```

end coder8a3;

7. Implemente un codificador de 8a3 con prioridad, enable y salida gs

entity coder8a3eg is

```
Port ( w : in std_logic_vector (7 downto 0);  
      e : in std_logic;  
      gs : out std_logic;  
      y : out std_logic_vector (2 downto 0));
```

end coder8a3eg;

8. Implemente un decodificador BCD a 7 segmentos. Use with - select.

entity myDeco7Seg is

Port (entrada : in std_logic_vector (3 downto 0);

a, b, c, d, e, f, g: out std_logic);

end myDeco7Seg;

entrada	a	b	c	d	e	f	g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	1	0	1	1
1010	1	1	1	0	1	1	1
1011	0	0	1	1	1	1	1
1100	1	1	1	1	0	0	0
1101	0	1	1	1	1	0	1
1110	1	0	0	1	1	1	1
1111	1	0	0	0	1	1	1

9. Implemente la siguiente tablas de la verdad.

op	r	b	a	ov		op	r	b	a	ov
0	0	0	0	0		1	0	0	0	0
0	0	0	1	0		1	0	0	1	0
0	0	1	0	0		1	0	1	0	1
0	0	1	1	1		1	0	1	1	0
0	1	0	0	1		1	1	0	0	0
0	1	0	1	0		1	1	0	1	1
0	1	1	0	0		1	1	1	0	0
0	1	1	1	0		1	1	1	1	0

```
entity calcOverflow is
Port ( a: in std_logic;
      b: in std_logic;
      r: in std_logic;
      op: in std_logic;
      ov: out std_logic);
end calcOverflow;
```

■ ■ ■

7. Generics; for - generate; componentes

Ejemplo

Implemente una and de N entradas y una or de M entradas

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ejemplo01 is
  generic (N: integer := 4;
           M: integer := 8);
  Port ( a : in std_logic_vector (N - 1 downto 0);
        b: in std_logic_vector (M - 1 downto 0);
        yOr : out std_logic;
        yAnd: out std_logic);
end ejemplo01 ;

architecture Behavioral of ejemplo01 is
  a_S : in std_logic_vector (N - 1 downto 0);
  b_S: in std_logic_vector (M - 1 downto 0);
begin

  a_s(0) <= a(0);
  andInst:
  for i in 1 to N - 1 generate
    begin
      a_s(i) <= a_s(i - 1) and a(i);
    end generate;

  yAnd <= a_S (N - 1);

  b_s(0) <= b(0);
  orInst:
  for i in 1 to M - 1 generate
    begin
      b_s(i) <= b_s(i - 1) or b(i);
    end generate;

  yOr <= b_S (M - 1);

end Behavioral;
```

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre `guia_07` e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. `xc7z010clg400-1`

1. Implemente un generador de paridad par de N bits.

```
entity paridadGen is
  Generic (N: integer := 4);
  Port ( a : in std_logic_vector (N - 1 downto 0);
        sel : in std_logic;
        p : out std_logic);
end paridadGen;
```

El bit de paridad se calcula de la siguiente forma:

- Paridad par: Se cuenta la cantidad de unos, si el total es par el bit de paridad vale 0, en caso contrario vale 1.
- Paridad impar: Se cuenta la cantidad de unos, si el total es impar el bit de paridad vale 0, en caso contrario vale 1

Por ejemplo para N = 4: $p \leq a(3) \text{ xor } a(2) \text{ xor } a(1) \text{ xor } a(0) \text{ xor } sel$;

entrada	Paridad par	Paridad impar
0000	0	1
0001	1	0
0101	0	1
1110	1	0

El bit de paridad se utiliza para detectar posibles errores en la transmisión de datos. Este se agrega al dato a transmitir y al ser recibido se puede verificar calculando la paridad del dato y comparándolo con el bit de paridad recibido.

2. Implemente un conversor gray a binario de N bits.

```
entity grayBinario is
  Generic (N: integer := 4);
  Port ( gray : in std_logic_vector (N-1 downto 0);
        binario : out std_logic_vector (N-1 downto 0));
end grayBinario;
```

Utilice las siguientes expresiones para realizar la conversión:

- $\text{binario}(i) = \text{gray}(i)$, para $i = N-1$
- $\text{binario}(i) = \text{binario}(i+1) \text{ xor } \text{gray}(i)$, para $(N-2) \leq i \leq 0$

La siguiente tabla muestra la conversión para 4 bits:

Binario	Gray		Binario	Gray
0000	0000		1000	1100
0001	0001		1001	1101
0010	0011		1010	1111
0011	0010		1011	1110
0100	0110		1100	1010
0101	0111		1101	1011
0110	0101		1110	1001
0111	0100		1111	1000

3. Implemente un conversor binario a gray de N bits utilizando for - generate.

entity binarioGray is

Generic (N: integer := 4);

Port (binario : in std_logic_vector (N-1 downto 0);

gray : out std_logic_vector (N-1 downto 0));

end binarioGray;

Utilice las siguientes expresiones para realizar la conversión:

- $\text{gray}(i) = \text{binario}(i)$, para $i = N-1$
- $\text{gray}(i) = \text{binario}(i+1) \text{ xor } \text{binario}(i)$, para $(N-2) \geq i \geq 0$

4. Utilizando como componente los dos VHDL creados en el punto anterior cree un conversor Binario a gray cuando la entrada sel toma el valor uno y gray a binario cuando la entrada sel toma el valor cero

entity conversorBinarioGray is

Generic (N: integer := 8);

Port (entrada: in std_logic_vector (N-1 downto 0);

salida : out std_logic_vector (N-1 downto 0);

sel : in std_logic;

end conversiorBinarioGray;



8. Circuitos aritméticos

Ejemplos

1. Ejemplo sumador de números signados

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity sumadorRestador is
generic (N : integer := 4);
port (a : in std_logic_vector(N-1 downto 0);
      b : in std_logic_vector(N-1 downto 0);
      sel : in std_logic;
      res : out std_logic_vector(N-1 downto 0));
end sumadorRestador ;

architecture Behavioral of sumadorRestador is

signal s : std_logic_vector(N-1 downto 0);
signal r : std_logic_vector(N-1 downto 0);

begin

s <= std_logic_vector(signed (b) + signed (a));
r <= std_logic_vector(signed (b) - signed (a));

res <= s when (sel = '1') else r;

end Behavioral;
```

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre guia_08 e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. xc7z010clg400-1

1. Implemente un comparador de magnitudes con salida mayor, menor e igual. Las salidas son activas nivel alto.

```
entity comparadorMag is
Generic (N: integer := 4);
Port ( a : in std_logic_vector (N-1 downto 0);
      b : in std_logic_vector (N-1 downto 0);
      aMayorQueB : out std_logic;
      aMenorQueB : out std_logic;
      al igualB : out std_logic);
end comparadorMag;
```

2. Implemente un comparador de números signados en CA2 con salida mayor, menor e igual. Las salidas son activas nivel alto.

```
entity comparadorCa2 is
Generic (N: integer := 4);
Port ( a : in std_logic_vector (N-1 downto 0);
      b : in std_logic_vector (N-1 downto 0);
```

```

    aMayorQueB : out std_logic;
    aMenorQueB : out std_logic;
    algualB : out std_logic;
end comparador;

```

3. Implemente un comparador que permita seleccionar con una entrada magCa2 si el circuito compara magnitudes (entrada en '0') o números signados en CA2 (entrada en '1'). Las salidas son activas nivel alto.

```

entity comparador is
Generic (N: integer := 4);
Port ( a : in std_logic_vector (N-1 downto 0);
      b : in std_logic_vector (N-1 downto 0);
      magCa2 : in std_logic;
      aMayorQueB : out std_logic;
      aMenorQueB : out std_logic;
      algualB : out std_logic;
end comparador;

```

4. Implemente un circuito que permite realice la suma/resta de dos números signados en CA2 de N bits.

- Si s_r vale 1, entonces resultado $\leq b + a$
- Si s_r vale 0, entonces resultado $\leq b - a$

```

entity sumadorRestador is
Generic (N: integer := 4);
Port ( a : in std_logic_vector (N-1 downto 0);
      b : in std_logic_vector (N-1 downto 0);
      s_r : in std_logic;
      ov : out std_logic;
      resultado: out std_logic_vector (N-1 downto 0));
end sumadorRestador;

```

5. Implemente un circuito que reciba un número signado en CA2 de 16 bits y sature el mismo a 8 bits.

```

entity sat16a8 is
Port ( entrada : in std_logic_vector (15 downto 0);
      salida : out std_logic_vector (7 downto 0));
end sat16a8;

```

6. Implemente un circuito que permite dividir una magnitud de 8 bits por 1, 2, 4, 8 según se seleccione. La parte decimal del resultado se trunca.

```

entity myDivMag8 is
Port ( entrada : in std_logic_vector (7 downto 0);
      div : in std_logic_vector (1 downto 0);
      salida : out std_logic_vector (7 downto 0));
end myDivMag8;

```

7. Implemente un circuito que permite dividir un número signado en convenio CA2 de 8 bits por 1, 2, 4, 8 según se seleccione. La parte decimal del resultado se trunca.

```

entity myDiv8 is
Port ( entrada : in std_logic_vector (7 downto 0);
      div : in std_logic_vector (1 downto 0);
      salida : out std_logic_vector (7 downto 0));
end myDiv8;

```

8. Implemente un circuito que realice operaciones aritméticas y lógicas

```

entity miniAlu is
generic (DATA_BITS: integer := 16);
port ( a: in std_logic_vector (DATA_BITS-1 downto 0);

```

```

b: in std_logic_vector (DATA_BITS-1 downto 0);
carryIn : in std_logic;
carryOut : out std_logic;
res: out std_logic_vector (DATA_BITS-1 downto 0);
sat: in std_logic;
code: in std_logic_vector (3 downto 0);
zero : out std_logic;
overflow : out std_logic;
negative : out std_logic);
end miniAlu;

```

Donde:

- a y b: son los operandos
- carryIn: Es el carry/borrow de entrada
- carryOut: Es el carry/borrow de salida
- res: es el resultado de las operaciones.
- sat: Si vale uno el resultado de la operación se satura.
- code: Indica qué operación debe realizar
- zero: Se coloca en uno cuando la res vale cero
- overflow: salida que indica si hubo overflow en la operación.
- negative: Indica si el resultado es negativo.

Code	Operación	Descripción
0000	$res = a \text{ and } b$	and bit a bit
0001	$res = a \text{ or } b$	or bit a bit
0010	$res = a \text{ xor } b$	xor bit a bit
0011	$(carry, res) = a + b + carry$	Suma con carry in y carry out Si el la entrada sat está en uno se satura el resultado de la operación
0100	$(borrow, res) = a - b + borrow$	Resta con borrow in y borrow out Si la entrada sat está en uno se satura el resultado de la operación
----	-	El resto de las combinaciones la salida vale cero

9. Circuitos secuenciales

Ejemplo

1. Implementación de un Latch D

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity myLatchD is
    Port ( ena : in std_logic;
          d   : in std_logic;
          q   : out std_logic);
end myLatchD;

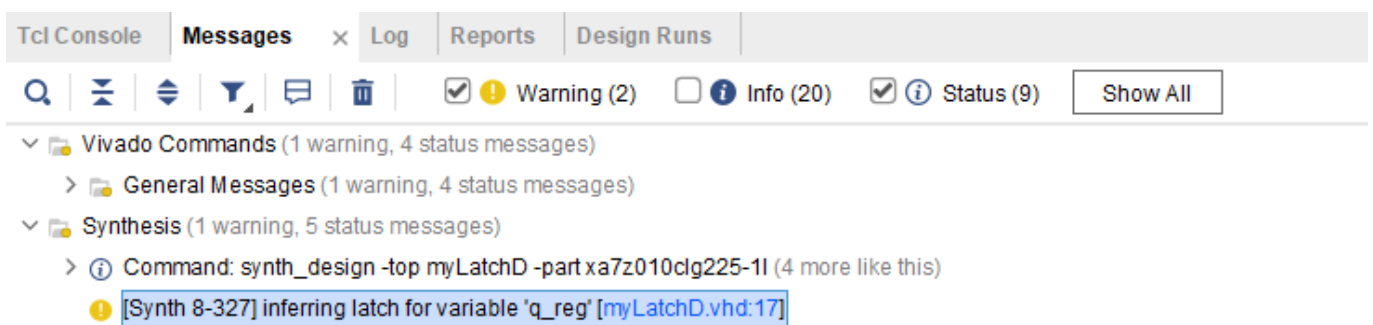
architecture Behavioral of myLatchD is

begin

process (ena, d)
begin
    if (ena = '1') then
        q <= d;
    end if;
end process;

end Behavioral;
```

Realice la síntesis del latch D y verifique los warning generado para identificar el Warning de inferencia de latch.



2. Flip-Flop D con reset sincrónico y clear (Reset asincrónico)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity myFFDR_Rasync is
    Port ( clk : in std_logic;
          rst : in std_logic;
          rstAsync : in std_logic;
          d : in std_logic;
```

```

        q : out std_logic);
end myFFDR_Rasync;

architecture Behavioral of myFFDR_Rasync is

begin

process (clk, rstAsync)
begin

    if (rstAsync = '1') then
        q <= '0';
    elsif (rising_edge (clk)) then
        if (rst = '1') then
            q <= '0';
        else
            q <= d;
        end if;
    end if;

end process;
end Behavioral;

```

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre `guia_09` e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. `xc7z010clg400-1`. Todos los process deben ser activos flanco ascendente, salvo indicación contraria. Todos los reset deben ser activos nivel alto (Resetea cuando la entrada vale '1'), salvo indicación contraria.

1. Implemente un latch D, con clear.

```

entity myLatchD_Clr is
Port ( ena : in std_logic;
      clr : in std_logic;
      d : in std_logic;
      q : out std_logic);
end myLatchD_Clr;

```

2. Implemente un flip-flop D con reset sincrónico.

```

entity myFFDR is
Port ( clk : in std_logic;
      rst : in std_logic;
      d : in std_logic;
      q : out std_logic);
end myFFDR;

```

3. Implemente un flip-flop D con reset sincrónico y enable.

```
entity myFFDREna is
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        d : in std_logic;
        q : out std_logic);
end myFFDREna;
```

4. Implemente un flip-flop D con reset sincrónico, enable y clear

```
entity myFFDREnaClr is
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        clr : in std_logic;
        d : in std_logic;
        q : out std_logic);
end myFFDREnaClr;
```

5. Implemente N flip-flop D con reset sincrónico y enable.

```
entity myFFDREN is
  Generic (N: integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        d : in std_logic_vector (N-1 downto 0);
        q : out std_logic_vector (N-1 downto 0));
end myFFDREN;
```

6. Implemente en VHDL flip-flop T con reset sincronico.

```
entity FFT is
  Port ( clk : in std_logic;
        rst : in std_logic;
        t : in std_logic;
        q : out std_logic);
end FFT;
```



10. Registros de desplazamiento

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre `guia_10` e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. `xc7z010clg400-1`. Todos los process deben ser activos flanco ascendente, salvo indicación contraria. Todos los reset deben ser activos nivel alto (Resetea cuando la entrada vale '1'), salvo indicación contraria.

1. Implemente un registro de desplazamiento de N bits con

- Reset síncronico.
- Enable
- Entrada serie (si), Salida serie (so)
- Entrada paralelo (pi) y entrada de carga paralelo (pl)
- Salida paralelo (po)

```
entity myShiftReg is
  Generic (N: integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        si : in std_logic;
        so : out std_logic;
        po : out std_logic_vector (N - 1 downto 0);
        pi : in std_logic_vector (N - 1 downto 0);
        pl : in std_logic);
end myShiftReg;
```

2. Implemente un contador en Johnson de N bits, con reset síncronico y enable

```
entity myCntJohnson is
  Generic (N: integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        q : out std_logic_vector (N - 1 downto 0));
end myCntJohnson;
```

3. Implemente un contador en anillo de N bits, con reset síncronico y enable

```
entity myCntRing is
  Generic (N: integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        q : out std_logic_vector (N - 1 downto 0));
end myCntRing;
```

4. Implemente un detector de flanco ascendente y descendente. Cuando se detecta el flanco correspondiente en la señal de entrada deberá poner la salida en uno por un ciclo de clock. Recuerde que `rising_edge` y `falling_edge` deben usarse sólo sobre una señal de clk

```
entity myEdgeDetector is
  Port ( clk : in std_logic;
        rst : in std_logic;
        d : in std_logic;
        ascendente : out std_logic;
```



```

    descendente : out std_logic);
end myEdgeDetector;

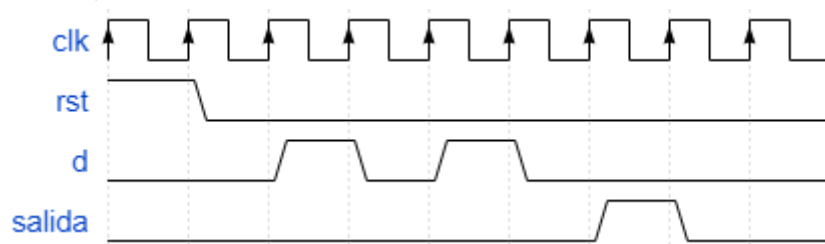
```

5. Implemente un circuito que coloque en uno su salida luego de detectar la secuencia "1010" que ingresa de forma serial por la entrada d. Cada bit de la secuencia dura un ciclo de clock. (Utilice un registro de desplazamiento)

```

entity detectorSecuencia is
Port ( clk : in  std_logic;
      rst : in  std_logic;
      d : in  std_logic;
      salida : out std_logic);
end detectorSecuencia;

```



6. Implemente un circuito que cada vez que la entrada gen reciba un pulso en uno por un ciclo de clock, genera la secuencia "1010" de forma serial a razón de un bit por ciclo de clock.

```

entity generadorSecuencia is
Port ( clk : in  std_logic;
      rst : in  std_logic;
      gen : in  std_logic;
      salida : out std_logic);
end generadorSecuencia;

```

7. Implemente en VHDL un Fibonacci LFSR de N bits (xor N entradas). Se recomienda usar xor_reduce para ello deberá agregar use ieee.std_logic_misc.all

```

entity lfsr_F is
Generic (N : integer := 4);
Port ( clk : in  std_logic;
      rst : in  std_logic;
      inits : in  std_logic_vector(N - 1 downto 0);
      taps : in  std_logic_vector(N - 1 downto 0);
      salida : out std_logic_vector(N - 1 downto 0));
end lfsr_F;

```

Donde:

- inits: Es el valor inicial que toma el LFRS (reset)
- taps: Cada elemento en uno del vector indica si forma ese bit parte de la realimentación.

8. Implemente en VHDL Galois LFSR de N bits (N xor de 2 entradas).

```

entity lfsr_G is
Generic (N : integer := 4);
Port ( clk : in  std_logic;
      rst : in  std_logic;
      inits : in  std_logic_vector(N - 1 downto 0);
      taps : in  std_logic_vector(N - 1 downto 0);
      salida : out std_logic_vector(N - 1 downto 0));
end lfsr_G;

```

Donde:

- inits: Es el valor inicial que toma el LFRS (reset)
- taps: Cada elemento en uno del vector indica si forma ese bit parte de la realimentación.

11. Contadores binarios

Ejemplo

1. Implementación de un contador binario de N bits con reset sincrónico y enable.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity myCntBinarioSimple is
  Generic (N: integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        q  : out std_logic_vector (N - 1 downto 0));
end myCntBinarioSimple;

architecture Behavioral of myCntBinarioSimple is
  signal cnt_S : unsigned (N - 1 downto 0);
begin

  process (clk)
  begin

    if (rising_edge (clk)) then
      if (rst = '1') then
        cnt_S <= (others => '0');
      elsif (ena = '1') then
        cnt_S <= cnt_S + 1;
      end if;
    end if;

  end process;

  q <= std_logic_vector (cnt_s);

end Behavioral;
```

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre `guia_11` e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. `xc7z010clg400-1`. Todos los process deben ser activos flanco ascendente, salvo indicación contraria. Todos los reset deben ser activos nivel alto (Resetea cuando la entrada vale '1'), salvo indicación contraria.

1. Implemente un contador en binario de N bits con:

- Reset síncronico
- Enable
- Entrada paralelo (d) y entrada de carga paralelo (dl)
- Salida paralelo (q)

```
entity myCntBinarioPI is
Generic (N: integer := 4);
Port ( clk : in std_logic;
      rst : in std_logic;
      ena : in std_logic;
      dl  : in std_logic;
      d   : in std_logic_vector (N - 1 downto 0);
      q   : out std_logic_vector (N - 1 downto 0));
end myCntBinarioPI;
```

2. Implemente un contador binario bidireccional de N bits.

- Si `dir = 0`, Ascendente
- Si `dir = 1`, Descendente

```
entity myCntBiDir is
Generic (N: integer := 4);
Port ( clk: in std_logic;
      rst: in std_logic;
      ena: in std_logic;
      dir: in std_logic;
      q : out std_logic_vector (N-1 downto 0));
end myCntBiDir;
```

3. Implemente un código VHDL que cada $M/2$ y M pulsos de clock ponga la salida en 1 durante 1 ciclo de clock.

```
entity myCnt is
Generic (M : integer := 100);
Port ( clk : in std_logic;
      rst : in std_logic;
      ena : in std_logic;
      salidaM_2 : out std_logic;
      salidaM : out std_logic);
end myCnt;
```

Coloque la siguiente línea de código en donde declara las signals para que se calcule automáticamente la cantidad de bits del contador en base a la cuenta a realizar:

```
constant N : integer := integer(ceil(log2(real (M))));
constant M_2 : unsigned (N-1 downto 0) := to_unsigned(M-1, N) / 2;
```

Deberá agregar el siguiente use: `use ieee.math_real.all`

4. Implemente un código VHDL que cada p pulsos de clock ponga la salida en 1 durante 1 ciclo de clock.

```
entity myCnt2 is
  Generic (N : integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        p  : in std_logic_vector (N - 1 downto 0);
        salida : out std_logic);
end myCnt2;
```

■ ■ ■

12. Máquinas de estados

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre `guia_12` e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. `xc7z010clg400-1`. Todos los process deben ser activos flanco ascendente, salvo indicación contraria. Todos los reset deben ser activos nivel alto (Resetea cuando la entrada vale '1'), salvo indicación contraria.

1. Implemente el siguiente código de una máquina de estados mealy y salida registrada.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ejemploMealy is
    Port ( clk : in std_logic;
          rst : in std_logic;
          entrada : in std_logic;
          salida : out std_logic;
          salidaR : out std_logic
        );
end ejemploMealy;

architecture Behavioral of ejemploMealy is
    type state_type is (stA, stB, stC , stD);
    signal state, next_state : state_type;
    signal salida_s : std_logic;

begin

estadoProc: process (clk)
begin
    if (rising_edge (clk)) then
        if (rst = '1') then
            state <= stA;
            salidaR <= '0';
        else
            state <= next_state;
            salidaR <= salida_s;
        end if;
    end if;
end process;

salida <= salida_s;
```

```

logicaSalida: process (state, entrada)
begin

    case (state) is
        when stA =>
            if entrada = '0' then
                salida_s <= '0';
            else
                salida_s <= '1';
            end if;
        when stB =>
            salida_s <= '1';
        when stC =>
            salida_s <= '1';
        when stD =>
            salida_s <= '0';
        when others =>
            salida_s <= '0';
    end case;
end process;

logicaEstadoFuturo: process (state, entrada)
begin

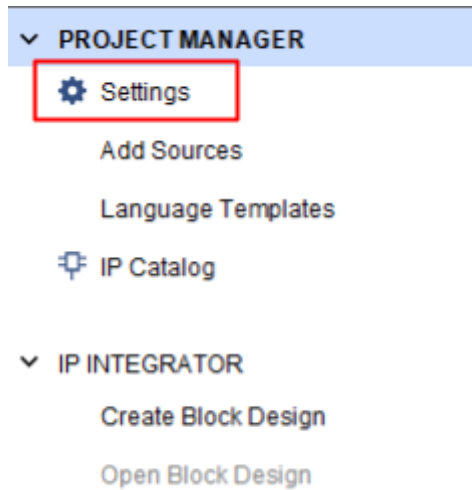
    next_state <= state;
    case (state) is
        when stA =>
            if entrada = '1' then
                next_state <= stB;
            end if;
        when stB =>
            next_state <= stC;
        when stC =>
            next_state <= stD;
        when stD =>
            next_state <= stA;
        when others =>
            next_state <= stA;
    end case;

end process;

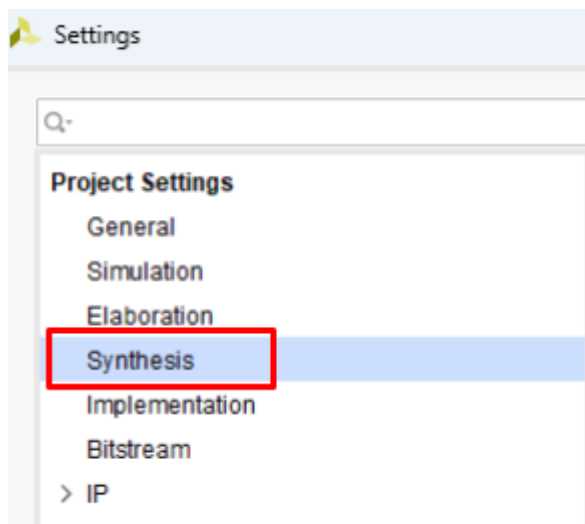
end Behavioral;

```

- a. Vaya a los Settings del proyecto



- b. En settings entre en las opciones de síntesis



- c. Buscar la opción **fsm extraction** y configurarla en **gray**

Synthesis

Specify various settings associated to Synthesis



Constraints

Default constraint set: constrs_1 (active)

Report Settings

Strategy: Vivado Synthesis Default Reports (Vivado Synthesis 2020)

Settings

Synth Design (vivado)		
tcl.pre		...
tcl.post		...
-flatten_hierarchy	rebuilt	▼
-gated_clock_conversion	off	▼
-bufg	12	
-directive	Default	▼
-retiming	☐	
-fsm_extraction*	gray	
-keep_equivalent_registers	off	
-resource_sharing	one_hot	
-control_set_opt_threshold	sequential	
-no_lc	johnson	
-no_srlextract	gray	
-shreg_min_size	auto	
	3	

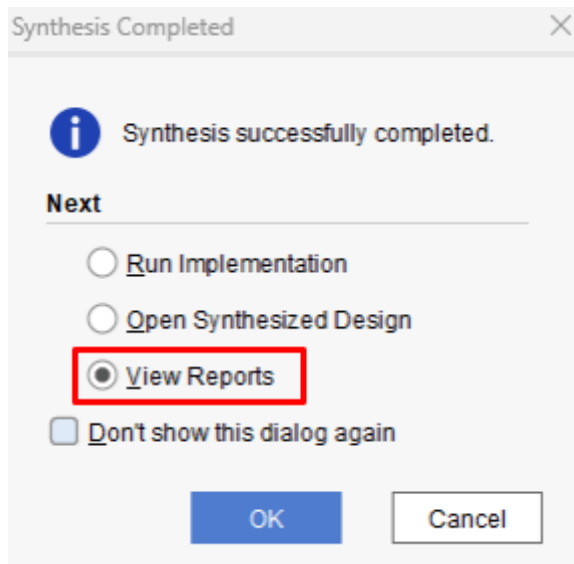
- d. Ejecute la síntesis

SYNTHESIS

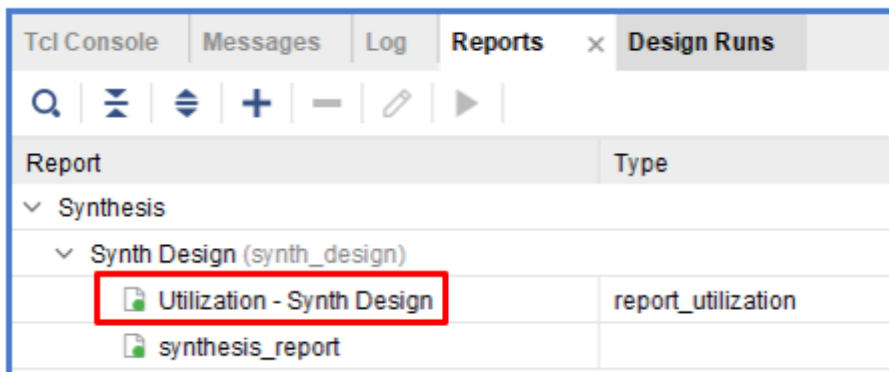
▶ Run Synthesis

> Open Synthesized Design

- e. Con la síntesis finalizada



- f. Vaya a la solapa de Reports, abra el reporte de Utilización

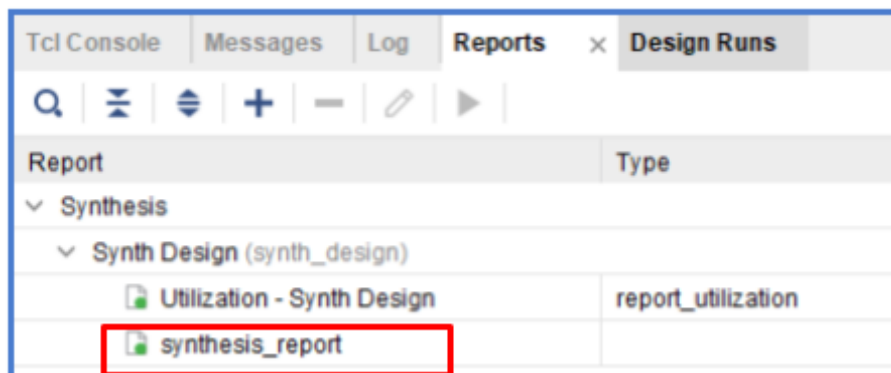


- g. En el reporte de utilización vaya al apartado 1 y observe los recursos utilizados, particularmente **Register as Flip Flop**

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	2	0	17600	0.01
LUT as Logic	2	0	17600	0.01
LUT as Memory	0	0	6000	0.00
Slice Registers	3	0	35200	<0.01
Register as Flip Flop	3	0	35200	<0.01
Register as Latch	0	0	35200	0.00
F7 Muxes	0	0	8800	0.00
F8 Muxes	0	0	4400	0.00

- h. Vaya a la solapa de Reports, abra el reporte de síntesis



- i. En el reporte de síntesis busque la tabla de asignación de estados

State	New Encoding	Previous Encoding
sta	00	00
stb	01	01
stc	11	10
std	10	11

- j. Repita los pasos desde el punto C cambiando la opción fsm extraction a One Hot, compare los resultados obtenidos de la síntesis particularmente los recursos usados y la asignación de estados.
2. Implemente el siguiente código de una máquina de estados Moore y verifique los reportes de síntesis y utilización

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ejemploMoore is
  Port ( clk : in std_logic;
        rst : in std_logic;
        entrada : in std_logic;
        salida : out std_logic;
        salidaR : out std_logic);
end ejemploMoore;

architecture Behavioral of ejemploMoore is
  type state_type is (stA, stB, stC , stD);
  signal state, next_state : state_type;
  signal salida_s : std_logic;

begin
```

```

estadoProc: process (clk)
begin
    if (rising_edge (clk)) then
        if (rst = '1') then
            state <= stA;
            salidaR <= '0';
        else
            state <= next_state;
            salidaR <= salida_s;
        end if;
    end if;
end process;

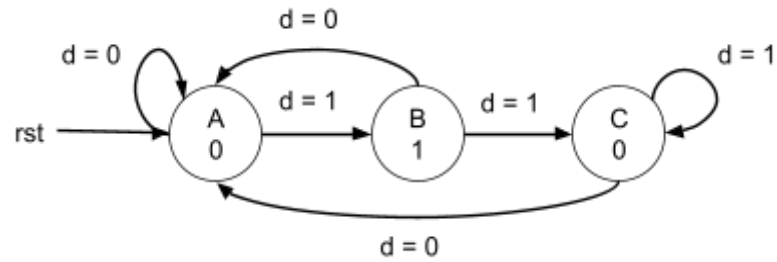
salida <= salida_s;

logicaSalida: process (state)
begin
    case (state) is
        when stA =>
            salida_s <= '0';
        when stB =>
            salida_s <= '1';
        when stC =>
            salida_s <= '1';
        when stD =>
            salida_s <= '1';
        when others =>
            salida_s <= '0';
    end case;
end process;

logicaEstadoFuturo: process (state, entrada)
begin
    next_state <= state;
    case (state) is
        when stA =>
            if entrada = '1' then
                next_state <= stB;
            end if;
        when stB =>
            next_state <= stC;
        when stC =>
            next_state <= stD;
        when stD =>
            next_state <= stA;
        when others =>
            next_state <= stA;
    end case;
end process;
end Behavioral;

```

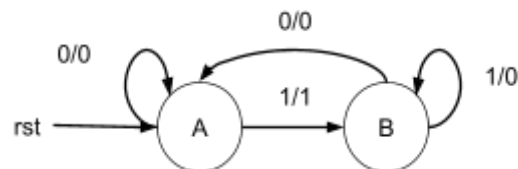
3. Implemente en VHDL el siguiente diagrama de estados



```

entity edgeMoore is
Port ( clk   : in std_logic;
      rst    : in std_logic;
      d      : in std_logic;
      y      : out std_logic);
end edgeMoore ;
  
```

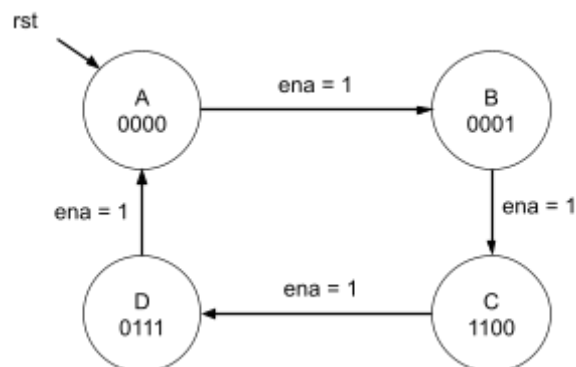
4. Implemente en VHDL el siguiente diagrama de estados.



```

entity edgeMealy is
Port ( clk   : in std_logic;
      rst    : in std_logic;
      d      : in std_logic;
      y      : out std_logic);
end edgeMealy ;
  
```

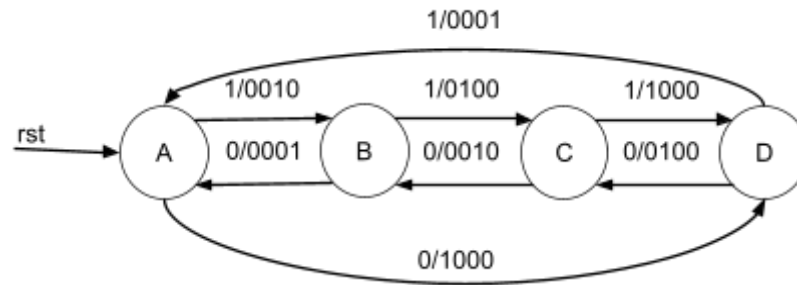
5. Implemente en VHDL el siguiente diagrama de estados. (si ena esta en cero permanece en el estado actual)



```

entity genSec is
Port ( clk   : in std_logic;
      rst    : in std_logic;
      ena    : in std_logic;
      y      : out std_logic_vector (3 downto 0));
end genSec ;
  
```

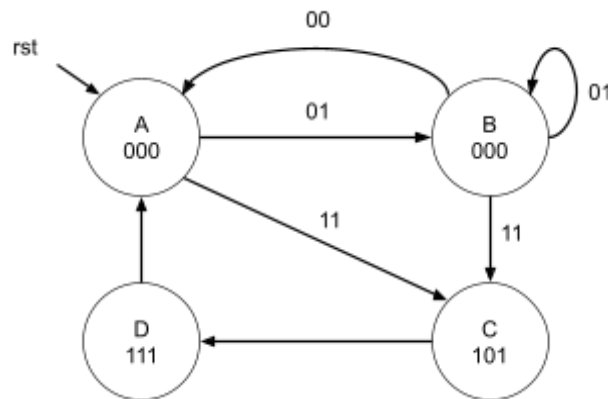
6. Implemente en VHDL el siguiente diagrama de estados



```

entity myCnt4b is
Port ( clk   : in std_logic;
      rst   : in std_logic;
      dir  : in std_logic;
      y: out std_logic_vector (3 downto 0));
end myCnt4b ;
  
```

7. Implemente en VHDL el siguiente diagrama de estados



```

entity genSec2 is
Port ( clk   : in std_logic;
      rst   : in std_logic;
      entradas : in std_logic_vector(1 downto 0);
      y: out std_logic_vector (2 downto 0));
end genSec2;
  
```

■ ■ ■

13. Testbench

Ejercicios

Cree un solo proyecto para todos los ejercicios con el nombre `guia_13` e implemente cada ejercicio colocando como nombre del archivo el nombre de la entidad. `xc7z010clg400-1`. Todos los process deben ser activos flanco ascendente, salvo indicación contraria. Todos los reset deben ser activos nivel alto (Resetea cuando la entrada vale '1'), salvo indicación contraria.

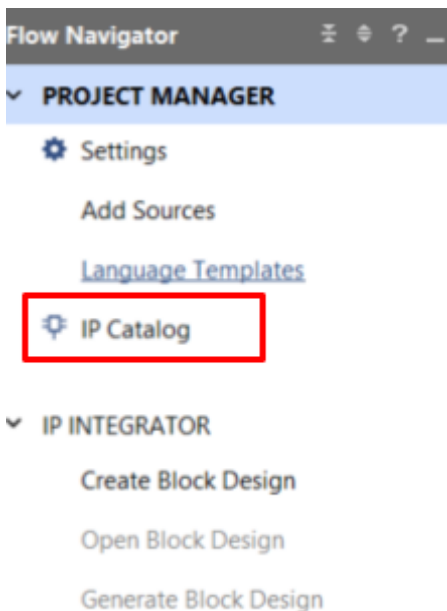
1. Implemente en VHDL una compuerta and de N entradas. Además realice el testbench que verifique su funcionamiento.

```
entity myAndN is
  Generic (N : integer := 4);
  Port ( a : in std_logic_vector (N - 1 downto 0);
        y : out std_logic);
end myAndN;
```

2. Implemente en VHDL N flip-flop T con reset síncrono y enable. Además realice el testbench que verifique su funcionamiento.

```
entity FFT_N is
  Generic (N : integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        ena : in std_logic;
        t : in std_logic_vector (N - 1 downto 0);
        q : out std_logic_vector (N - 1 downto 0));
end FFT;
```

3. Instancie una block ram como se muestra a continuación y luego realice un testbench para verificar su funcionamiento. Ayúdese con las formas de onda provistas.
 - a. Abra el IP Catalog



b. Dentro de Vivado repository->Basic element encontrará la Block Ram

Project Summary x IP Catalog x

Cores | Interfaces

Search:

Name	AXI4	Status	License	VLNV
▼ Vivado Repository				
> Alliance Partners				
> Audio Connectivity & Processing				
> Automotive & Industrial				
> AXI Infrastructure				
> AXIS Infrastructure				
> BaseIP				
▼ Basic Elements				
> Accumulators				
Concat		Production	Included	xilinx.com:ip:xlconcat:2.1
Constant		Production	Included	xilinx.com:ip:xlconstant:1.1
> Counters				
DSP48 Macro		Production	Included	xilinx.com:ip:xbip_dsp48_macro:3.0
DSP Macro		Production	Included	xilinx.com:ip:dsp_macro:1.0
▼ Memory Elements				
Block Memory Generator	AXI4	Production	Included	xilinx.com:ip:blk_mem_gen:8.4
Distributed Memory Generator		Production	Included	xilinx.com:ip:dist_mem_gen:8.0
oddr		Production	Included	xilinx.com:ip:oddr:1.0

- c. Configure la solapa Basic mo se muestra a continuación (Coloque como nombre del componente blockRamTest0)

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

|| + BRAM_PORTA

Component Name blockRamTest0

Basic	Port A Options	Other Options	Summary
Interface Type Native Memory Type Single Port RAM ☐ Generate address interface with 32 bits ☐ Common Clock			
ECC Options ECC Type No ECC ☐ Error Injection Pins Single Bit Error Injection			
Write Enable ☐ Byte Write Enable Byte Size (bits) 9			
Algorithm Options Defines the algorithm used to concatenate the block RAM primitives. Refer datasheet for more information. Algorithm Minimum Area Primitive 8kx2			

OK Cancel

d. Configure la solapa port como se ve a continuación

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

+ BRAM_PORTA

Component Name blockRamTest0

Basic Port A Options Other Options Summary

Memory Size

Write Width 16 Range: 1 to 4608 (bits)

Read Width 16

Write Depth 1024 Range: 2 to 1048576

Read Depth 1024

Operating Mode Write First Enable Port Type Always Enabled

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

Port A Output Reset Options

☐ RSTA Pin (set/reset pin) Output Reset Value (Hex) 0

☐ Reset Memory Latch Reset Priority CE (Latch or Register Enable)

READ Address Change A

☐ Read Address Change A

OK Cancel

e. En la solapa other option agregue el archivo .coe provisto

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

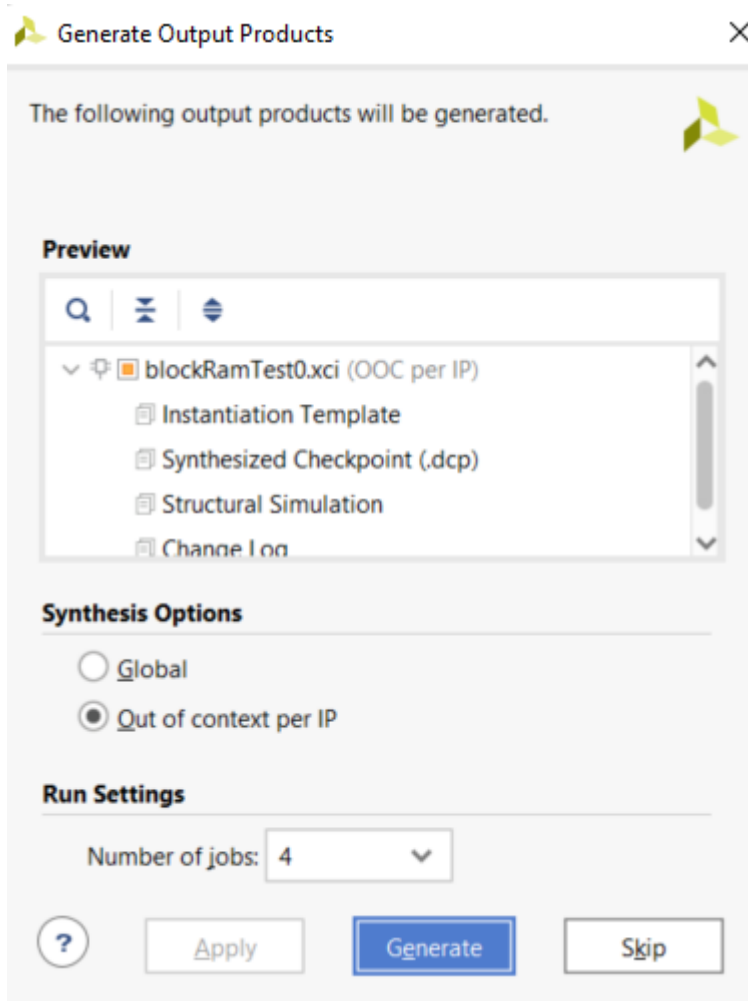
|| + BRAM_PORTA

Component Name blockRamTest0

Basic	Port A Options	Other Options	Summary
Pipeline Stages within Mux 0 Mux Size: 1x1			
Memory Initialization			
☒ Load Init File			
Coe File blockRamTb.coe		Browse	Edit
☒ Fill Remaining Memory Locations			
Remaining Memory Locations (Hex)		5555	
Structural/UniSim Simulation Model Options			
Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.			
Collision Warnings		All	
Behavioral Simulation Model Options			
☐ Disable Collision Warnings ☐ Disable Out of Range Warnings			

OK Cancel

- f. Una vez configurada toda la Block Ram presione el botón OK y en la siguiente ventana presione el botón Generate



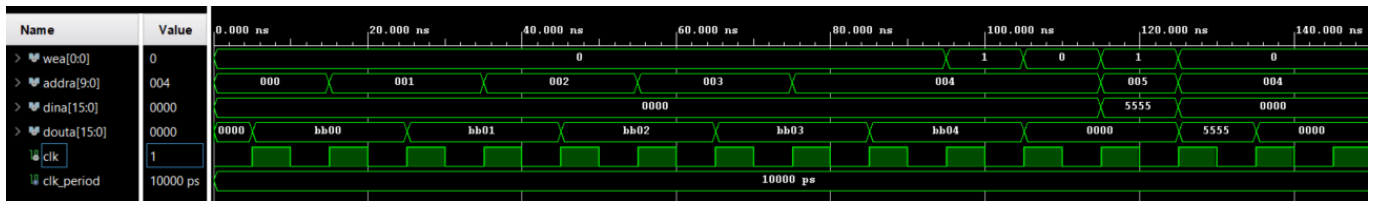
- g. Cree un testbench instanciando la Block Ram, la declaración para poder instanciar la block ram en el testbench es la siguiente

```
component blockRamTest0
port (
  clka : in std_logic;
  wea : in std_logic_vector(0 DOWNTO 0);
  addra : in std_logic_vector(9 DOWNTO 0);
  dina : in std_logic_vector(15 DOWNTO 0);
  douta : out std_logic_vector(15 DOWNTO 0)
);
end component;
```

Donde:

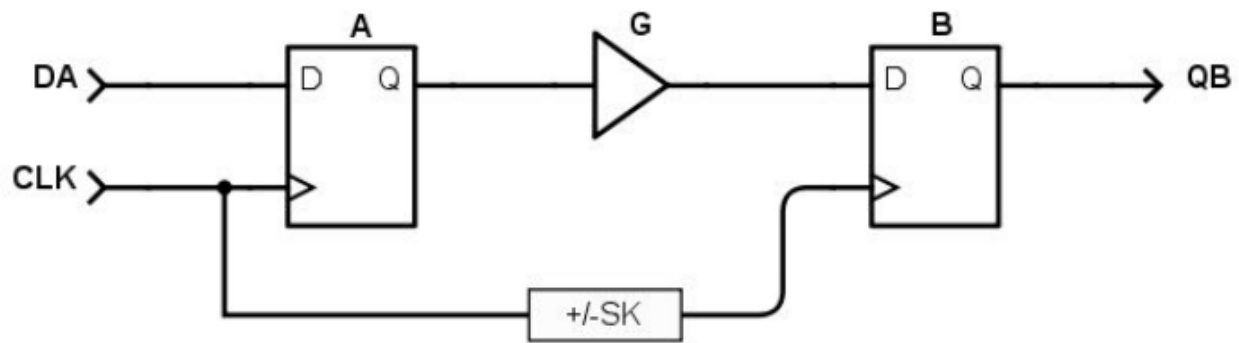
- clka: Es el clock de la block ram. Es activo flanco ascendente.
- wea: Es la habilitación para escribir en la memoria. Es activo alto. Para lograr una escritura debe mantenerse al menos un ciclo de clock en alto.
- addra: Dirección donde se escribe o lee en la memoria.
- dina: Dato a escribir en la dirección addra de la memoria.
- douta: Dato leído de la dirección addra de la memoria

- h. En el testbench implemente los process necesarios para generar las formas de onda como se observan en la siguiente imagen



14. Timing

1. Dado el siguiente circuito, parámetros de flip-flops y lógica combinacional



FFA	Min[ns]	Max[ns]
t_{cq}	2	3
t_s	0.2	4
t_h	0.1	0.5

FFB	Min[ns]	Max[ns]
t_{cq}	2	3
t_s	0.2	4
t_h	0.1	0.5

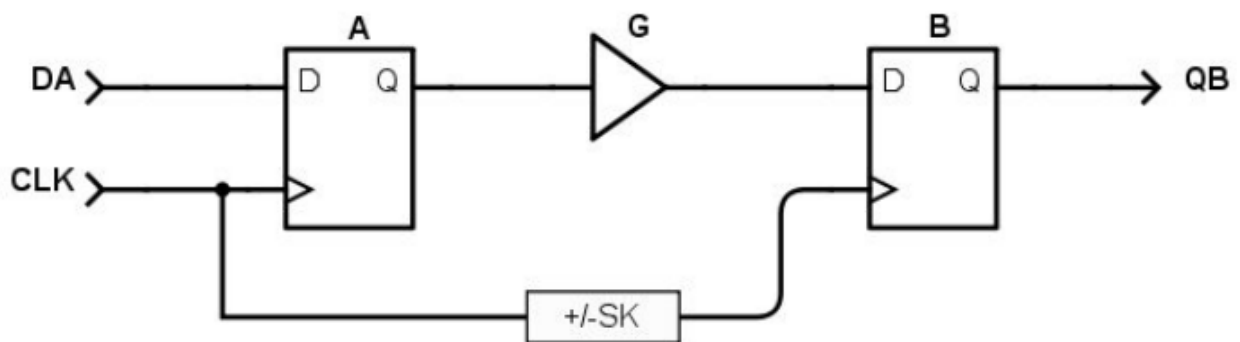
gate	Min[ns]	Max[ns]
t_g	1	2

Halle:

- Frecuencia máxima de clock para $t_{sk} = 0$
- t_{su_slack} y t_{h_slack} para $t_{sk} = 0$ y $F = 100$ MHz
- El skew máximo tolerable por el circuito para una Frecuencia de clock de 100 MHz
- t_{su_slack} y t_{h_slack} para $t_{sk} = 2$ ns y $F = 100$ MHz

Justifique las respuestas utilizando diagramas temporales.

2. Dado el siguiente circuito, parámetros de flip-flops y lógica combinacional



FFA	Min[ns]	Max[ns]
t_{cq}	1	2
t_s	0.2	4
t_h	0.1	0.5

FFB	Min[ns]	Max[ns]
t_{cq}	1	4
t_s	0.2	4
t_h	0.1	0.5

gate	Min[ns]	Max[ns]
t_g	1	2

Halle:

- 
- a. Frecuencia máxima de clock para $t_{sk} = 0$
 - b. t_{su_slack} y t_{h_slack} para $t_{sk} = 0$ y $F = 100$ MHz
 - c. El skew máximo tolerable por el circuito para una Frecuencia de clock de 100 MHz
 - d. t_{su_slack} y t_{h_slack} para $t_{sk} = 2$ ns y $F = 100$ MHz

Justifique las respuestas utilizando diagramas temporales.

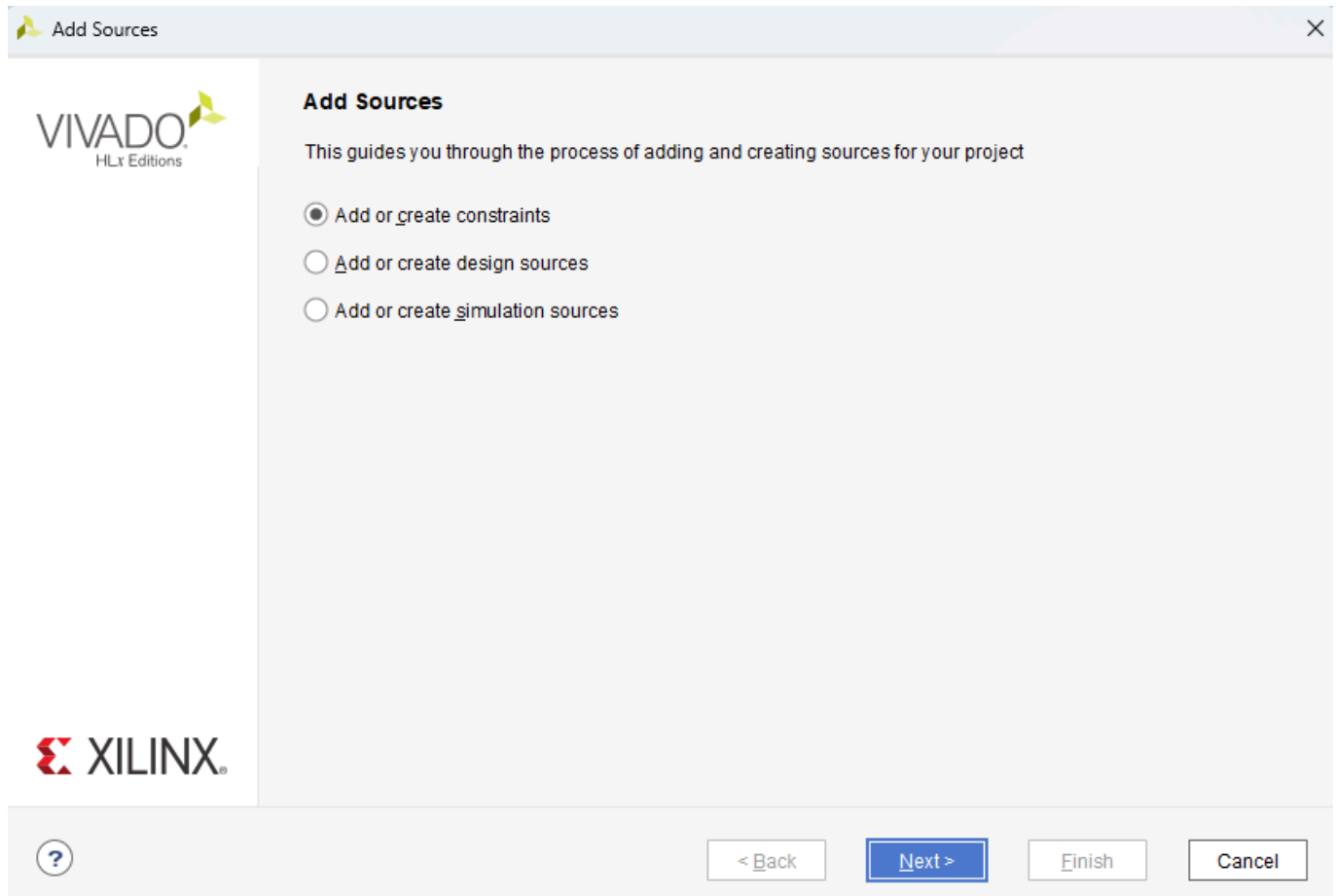


15. Ejercicios integradores I.

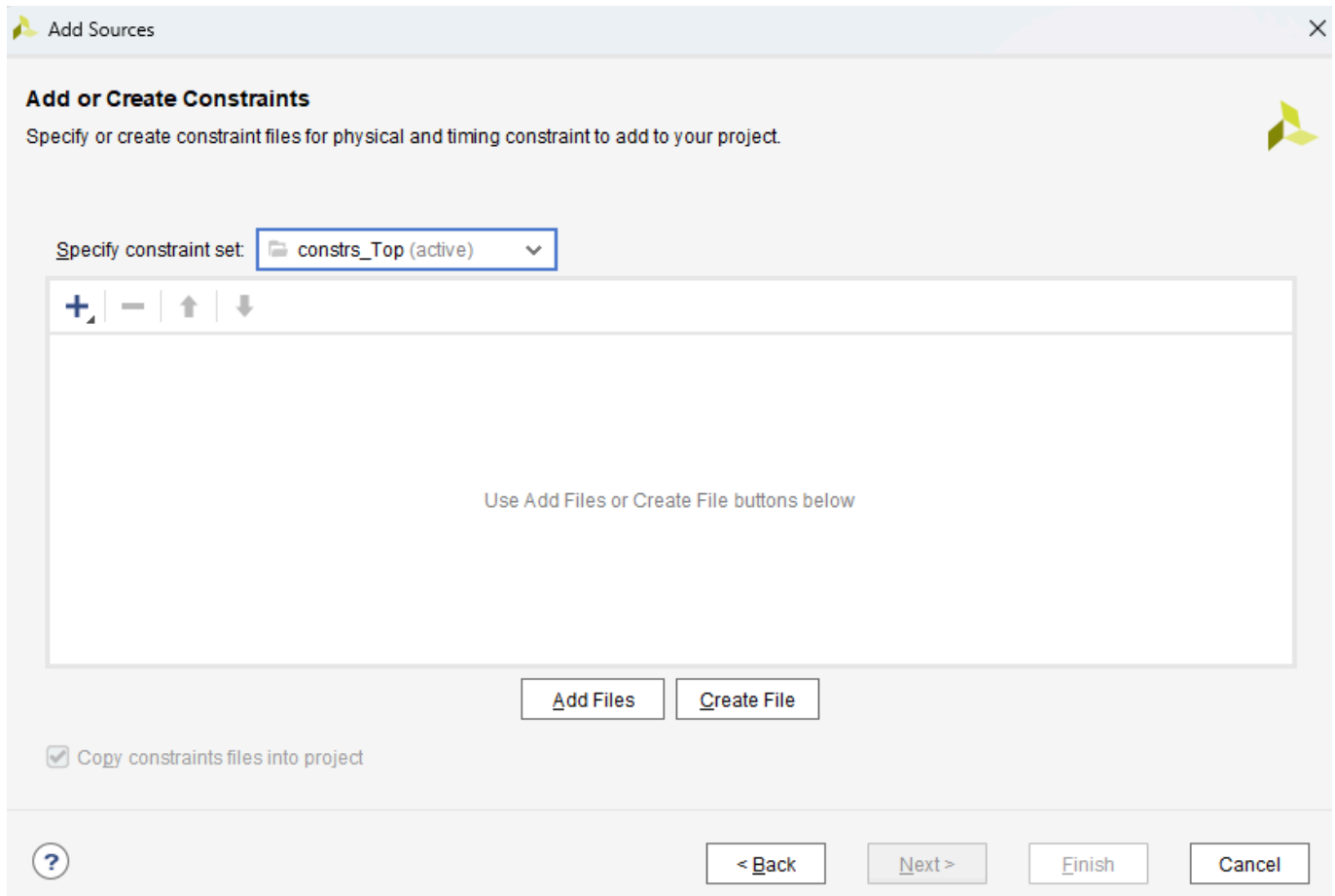
Para todos los proyectos de este capítulo utilice el siguiente dispositivo xc7z010clg400-1. Los siguientes pasos lo guiarán en la realización de la síntesis de un diseño en VHDL incluyendo las restricciones temporales y las restricciones de I/O.

Ejemplo de implementación de un LFSR

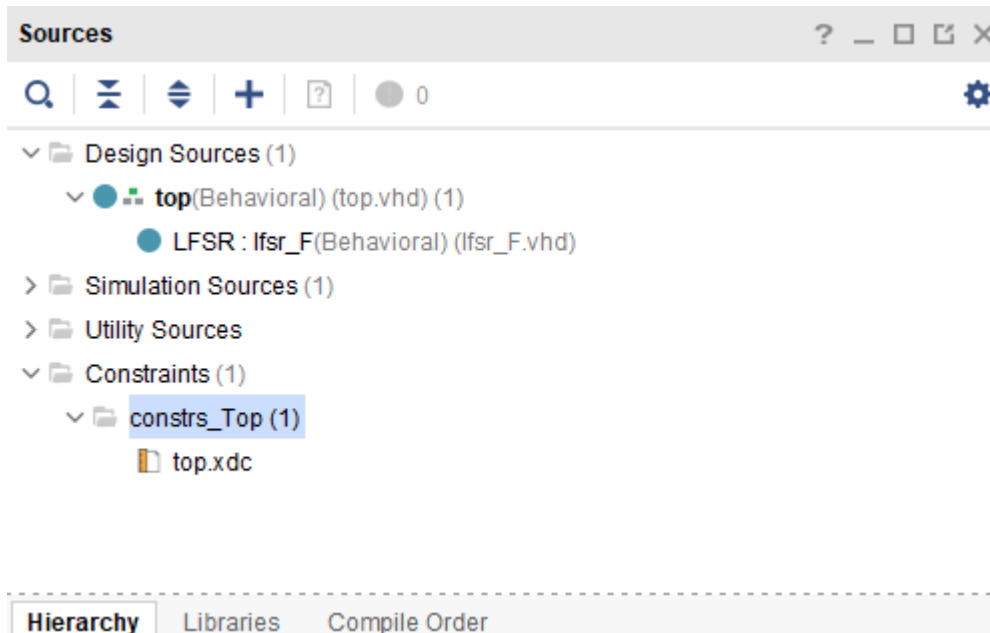
1. Cree un proyecto en Vivado con el LFSR_F (LFRS de Fibonacci) y agregue el archivo top.vhdl
2. Incluya el el archivo de restricciones top.xdc, para ello vaya a File->Add source, luego seleccione en la siguiente ventana la opción Add or create constraints y presione next



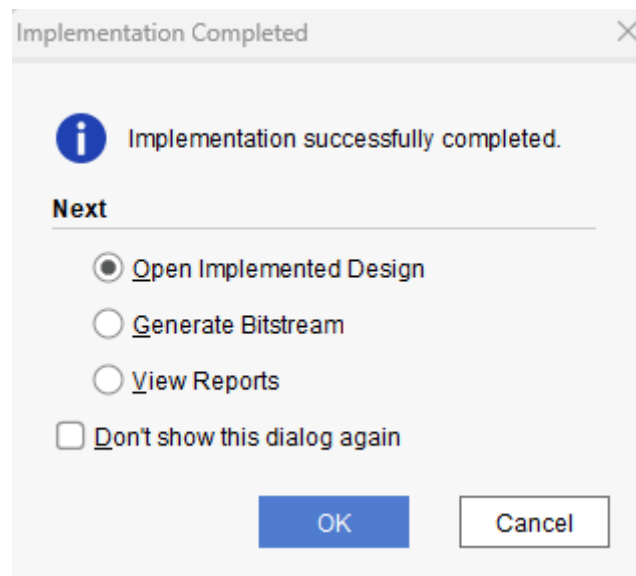
En esta ventana deberá agregar el archivo de restricciones, para luego presionar finish



Al finalizar el proyecto deberá quedar como se muestra a continuación



- Realice la implementación, para ello vaya al menú Flow->Run Implementation Design. Al terminar el proceso aparecerá la siguiente ventana en la cual deberá seleccionar la opción Open Implementation Design.



- Al finalizar la Implementación vea la solapa de timing que aparecerá en la ventana inferior donde podrá comprobar si los requerimientos temporales se cumplieron.

Tcl Console

Messages

Log

Reports

Design Runs

DRC

Methodology

Power

Timing

x

Q

≡

⌵

●

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

> Check Timing (2)

> Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.618 ns	Worst Hold Slack (WHS): 0.164 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1025	Total Number of Endpoints: 1025	Total Number of Endpoints: 1026

All user specified timing constraints are met.

En este caso se observa que ambos slacks son positivos.

Setup	Hold
Worst Negative Slack (WNS): 4.618 ns	Worst Hold Slack (WHS): 0.164 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1025	Total Number of Endpoints: 1025
All user specified timing constraints are met.	

- Modificamos el archivo de restricciones top.xdc para que el diseño funcione con un clock de 250MHz con un ciclo de actividad del 50%, para ello abra el archivo top.xdc y reemplace la línea

```
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports clk]
```

por

```
create_clock -period 4.000 -name sys_clk_pin -waveform {0.000 2.000} -add [get_ports clk]
```

Tcl Console	Messages	Log	Reports	Design Runs	DRC	Methodology	Power	Timing	x
Q ≡ ⚙ ❗ Design Timing Summary General Information Timer Settings ❗ Design Timing Summary Clock Summary (1) > Check Timing (2) > Intra-Clock Paths Inter-Clock Paths Other Path Groups Setup Hold Pulse Width Worst Negative Slack (WNS): -0.438 ns Worst Hold Slack (WHS): 0.117 ns Worst Pulse Width Slack (WPWS): 1.500 ns Total Negative Slack (TNS): -0.438 ns Total Hold Slack (THS): 0.000 ns Total Pulse Width Negative Slack (TPWS): 0.000 ns Number of Failing Endpoints: 1 Number of Failing Endpoints: 0 Number of Failing Endpoints: 0 Total Number of Endpoints: 1026 Total Number of Endpoints: 1026 Total Number of Endpoints: 1027 Timing constraints are not met.									

En este caso se observa que el diseño no cumple los requerimientos temporales, siendo el slack de setup negativo.

Setup	Hold
Worst Negative Slack (WNS): -0.438 ns	Worst Hold Slack (WHS): 0.117 ns
Total Negative Slack (TNS): -0.438 ns	Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 1	Number of Failing Endpoints: 0
Total Number of Endpoints: 1026	Total Number of Endpoints: 1026

Timing constraints are not met.

Si presiona sobre el número del slack de setup positivo se abrirá la siguiente ventana donde se observa en rojo el camino que posee el slack de setup negativo.

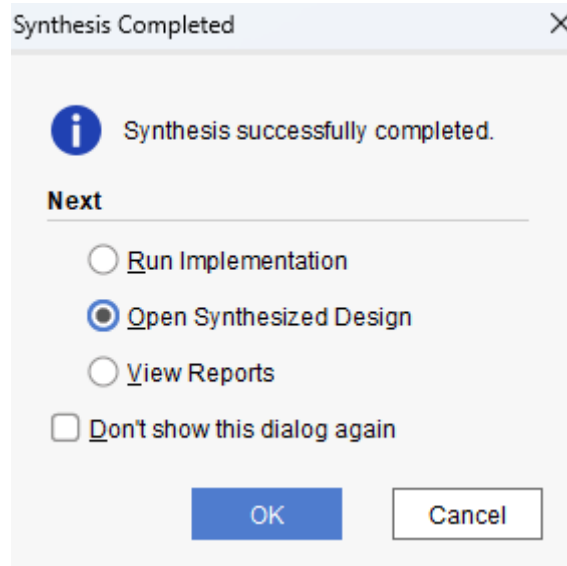
Name	Slack ¹	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	-0.438	4	2	LFSR/salida_S_reg[424]/C	LFSR/salida_S_reg[0]/D	4.446	1.014	3.432	4.0
Path 2	2.015	0	2	LFSR/salida_S_reg[263]/C	LFSR/salida_S_reg[264]/D	1.576	0.419	1.157	4.0
Path 3	2.058	0	2	LFSR/salida_S_reg[572]/C	LFSR/salida_S_reg[573]/D	1.537	0.419	1.118	4.0
Path 4	2.118	0	2	LFSR/salida_S_reg[911]/C	LFSR/salida_S_reg[912]/D	1.586	0.419	1.167	4.0
Path 5	2.155	0	2	LFSR/salida_S_reg[233]/C	LFSR/salida_S_reg[234]/D	1.592	0.419	1.173	4.0
Path 6	2.199	0	2	LFSR/salida_S_reg[857]/C	LFSR/salida_S_reg[858]/D	1.364	0.419	0.945	4.0
Path 7	2.226	0	2	LFSR/salida_S_reg[767]/C	LFSR/salida_S_reg[768]/D	1.439	0.478	0.961	4.0

Realizando doble clic sobre cualquiera path

Summary	
Name	Path 1
Slack	-0.438ns
Source	LFSR/salida_S_reg[424]/C (rising edge-triggered cell FDSE clocked by sys_clk_pin {rise@0.000ns fall@2.000ns period=4.000ns})
Destination	LFSR/salida_S_reg[0]/D (rising edge-triggered cell FDSE clocked by sys_clk_pin {rise@0.000ns fall@2.000ns period=4.000ns})
Path Group	sys_clk_pin
Path Type	Setup (Max at Slow Process Corner)
Requirement	4.000ns (sys_clk_pin rise@4.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay	4.446ns (logic 1.014ns (22.807%) route 3.432ns (77.193%))
Logic Levels	4 (LUT6=4)
Clock Path Skew	0.014ns
Clock Uncertainty	0.035ns

Ejemplos warning latch

1. Cree un proyecto en Vivado con el myLatch_D (Un latch D)
2. Incluya el archivo de restricciones myLatch_D.xdc, para ello vaya a File->Add source, luego seleccione en la siguiente ventana la opción Add or create constraints y presione next
3. Ejecute la síntesis presionando la tecla F11 y al finalizar abra el diseño sintetizado.



4. Observe la advertencia en la ventana de mensajes sobre la inferencia de un Latch. En este caso, es correcto, ya que es lo que hemos implementado. Sin embargo, en otros diseños donde no se utilice un latch, debes verificar y eliminar cualquier inferencia de latch que pueda ocurrir.



Es importante prestar atención a estas advertencias, ya que la inferencia de un latch puede tener consecuencias no deseadas en el funcionamiento del circuito. Los latches pueden introducir retrasos y consumir más recursos de los necesarios, lo que puede afectar negativamente el rendimiento y la eficiencia del diseño.

Por lo tanto, es recomendable revisar cuidadosamente el código y utilizar técnicas de diseño adecuadas para evitar la generación de latches no deseados. Esto implica asegurarse de que todas las señales de entrada estén asignadas en todos los caminos posibles dentro de los procesos, y utilizar las estructuras if-else o case, adecuadamente para evitar la inferencia involuntaria de latches.

Recuerde que es importante comprender la funcionalidad de los latches y su uso adecuado en el diseño de circuitos. Siempre busca optimizar y mejorar tus diseños para garantizar un funcionamiento correcto y eficiente.

Ejercicios

1. Implemente un circuito que cuente la cantidad de flancos ascendentes de la señal de entrada d

```
entity myEdgeCnt is
  Generic (N: integer := 8);
  Port ( clk: in std_logic;
        rst: in std_logic;
        d: in std_logic;
        q : out std_logic_vector (N-1 downto 0));
end myEdgeCnt;
```

- Implemente en VHDL el diseño y realice la verificación del mismo con un testbench
- Verifique el funcionamiento del circuito para las siguientes configuraciones
 - Fclk = 200MHz; N = 8
 - Fclk = 200MHz; N = 4

2. Implemente un circuito que haga parpadear una salida con una cadencia de 1 segundo, suponiendo que el clock del sistema es el configurado en el generic SYS_CLK. Se recomienda utilizar como componente uno de los contadores implementados anteriormente. Realice el testbench suponiendo un clock de sistema menor para agilizar la simulación.

```
entity parpadeo1S is
  Generic (SYS_CLK: integer := 100000000);
  Port ( clk : in std_logic;
        rst : in std_logic;
        salida : out std_logic);
end parpadeo1S;
```

- Implemente en VHDL el diseño y realice la verificación del mismo con un testbench, pero modifique SYS_CLK adecuadamente para ver el parpadeo en un tiempo razonable de simulación.
- Verifique el funcionamiento del circuito para las siguientes configuraciones
 - Fclk = 100MHz
 - Fclk = 200MHz

3. Implemente en VHDL utilizando máquinas de estado un circuito que controle un contador de N bits. Este circuito dispondrá de dos entradas, start y stop. Ambas entradas son activas nivel alto y solo se activan por un ciclo de clock. El comportamiento del circuito es el siguiente:

- Si se activa start el contador comienza a contar hasta que se active stop, mientras tanto se ignoran otras activaciones de la entrada start.
- Si el contador está contando se detendrá solo al activarse la entrada stop.
- Si el contador está detenido y se activa la entrada stop el contador vuelve a cero.
- Si el contador está detenido y se activa la entrada start el contador continúa contando.

La entidad del contador a usar es la siguiente:

```
entity myCntBiDir is
  Generic (N: integer := 4);
  Port ( clk: in std_logic;
        rst: in std_logic;
        ena: in std_logic;
        dir: in std_logic;
        q : out std_logic_vector (N-1 downto 0));
end myCntBiDir;
```

A continuación la entidad del circuito a implementar:

```
entity stopWatch is
  Generic (N: integer := 4);
  Port ( clk : in std_logic;
        rst : in std_logic;
        start : in std_logic;
        stop : in std_logic;
        salida: out std_logic_vector (N - 1 downto 0));
end stopWatch;
```

- a. *Implemente en VHDL el diseño y realice la verificación del mismo con un testbench.*
 - b. *Verifique el funcionamiento del circuito para las siguientes configuraciones*
 - a. *Fclk = 100MHz*
 - b. *Fclk = 200MHz*
4. Implemente un contador de N bits que cuente según la serie de Fibonacci. Puede considerar que N siempre será mayor o igual que 4. Por ejemplo para N = 5 la secuencia debe ser 0, 1, 1, 2, 3, 5, 8, 13, 21, 0, 1, 1, 2, 3, 5, 8 ...

```
entity fibonacciCnt is
  Generic (N: integer := 5);
  Port ( clk: in std_logic;
        rst: in std_logic;
        ena: in std_logic;
        cnt: in std_logic_vector (N - 1 downto 0));
end fibonacciCnt;
```

Se pide:

- Un diagrama en bloques del VHDL implementado
- La descripción en VHDL.
- Testbench que demuestre el funcionamiento.

5. Realice la descripción de un circuito en VHDL que recibe un dato de 8 bits en el flanco ascendente de la señal sck y realice una descompresión a 13 bits según la siguiente tabla. El resultado de la descompresión debe mostrarse en el flanco descendente de la señal sck. Se sabe que la frecuencia de la señal sck es de 1MHz.

Entrada								Entrada												
S	0	0	0	A	B	C	D	S	0	0	0	0	0	0	0	0	A	B	C	D
S	0	0	1	A	B	C	D	S	0	0	0	0	0	0	1	A	B	C	D	-
S	0	1	0	A	B	C	D	S	0	0	0	0	0	1	A	B	C	D	-	-
S	0	1	1	A	B	C	D	S	0	0	0	0	1	A	B	C	D	-	-	-
S	1	0	0	A	B	C	D	S	0	0	0	1	A	B	C	D	-	-	-	-
S	1	0	1	A	B	C	D	S	0	0	1	A	B	C	D	-	-	-	-	-
S	1	1	0	A	B	C	D	S	0	1	A	B	C	D	-	-	-	-	-	-
S	1	1	1	A	B	C	D	S	1	A	B	C	D	-	-	-	-	-	-	-

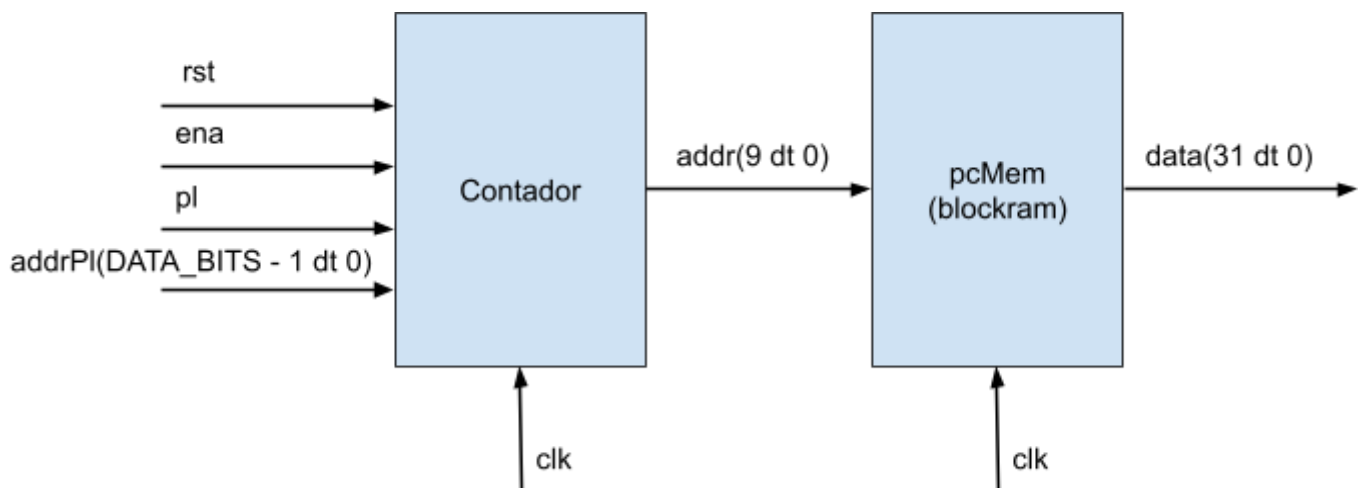
```

entity companding_A is
    Port ( clk : in std_logic;
          rst : in std_logic;
          sck : in std_logic;
          entrada : in STD_LOGIC_VECTOR (12 downto 0);
          salida : out STD_LOGIC_VECTOR (7 downto 0));
end companding_A;

```

- Dibuje el circuito a nivel de compuertas y flip-flop (use un FFD)
- Realice la descripción en VHDL
- Realice un testbench que demuestre su funcionamiento

6. Implemente en VHDL el siguiente diagrama de bloques que consiste en un contador y un blockram



La configuración de la blockram con nombre pcMem se observa en las dos imágenes que se muestran a continuación:

The images show the configuration of the pcMem component in a design tool. The top-left screenshot shows the 'Basic' tab with settings for Interface Type (Native), Memory Type (Single Port ROM), ECC Options (No ECC), Write Enable (Byte Write Enable), and Algorithm Options (Minimum Area, 8x2 Primitive). The top-right screenshot shows the 'Port A Options' tab with Memory Size (Port A Width: 32, Port A Depth: 1024), Operating Mode (Write First), and Port A Optional Output Registers. The bottom-left screenshot shows the 'Other Options' tab with Pipeline Stages within Mux (0), Memory Initialization (Load Init File, Coe File: /pcTest.coe), and Structural/UniSim Simulation Model Options.

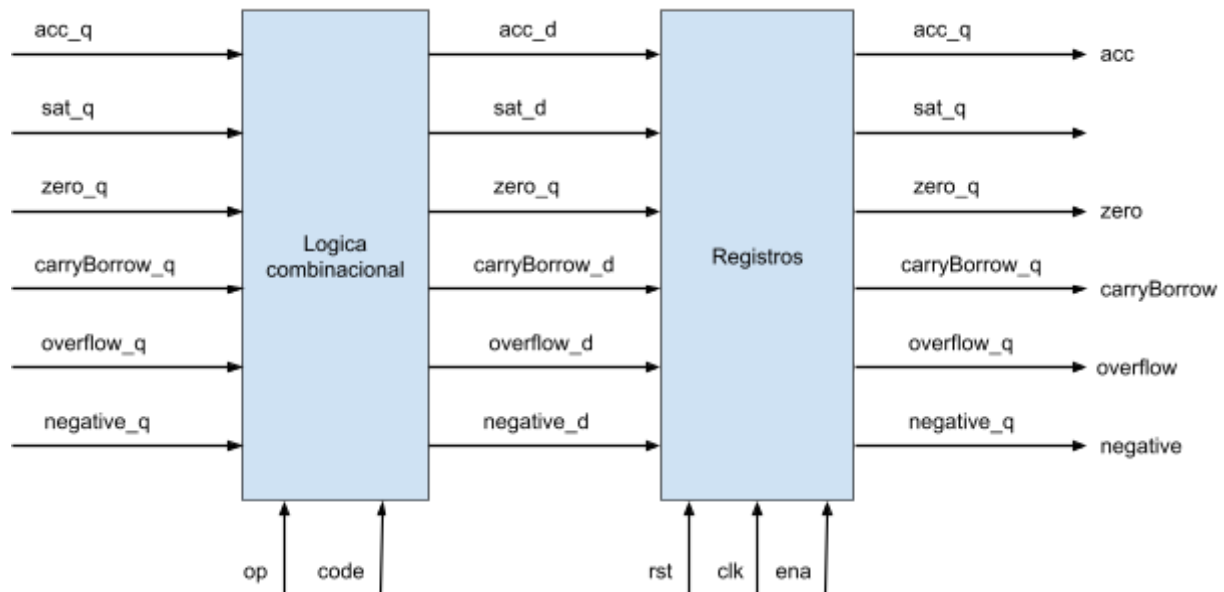
La entidad del diseño es la siguiente
entity pc is

```
Generic ( DATA_BITS : integer := 16);
Port ( clk : in std_logic;
      rst : in std_logic;
      ena : in std_logic;
      pl : in std_logic;
      plAddr : in std_logic_vector (DATA_BITS - 1 downto 0);
      data : out std_logic_vector (31 downto 0));
end pc;
```

Donde:

- clk: Clock del sistema
- rst: reset del sistema
- enable: Cuando esta en '1' se incrementa el address de la memoria en 1 (la salida del contador)
- pl: Al colocarse en '1' carga de forma paralela la entrada plAddr cómo address de la memoria (la salida del contador)
- plAddr: Dirección a cargar de la memoria al activarse pl.
- data: Salida de datos de la memoria.

7. Implemente una ALU que posea un registro acumulador y realice las operaciones descritas en la tabla que se encuentra debajo. El diagrama en bloques de la ALU es el siguiente.



La Unidad Aritmético-Lógica (ALU) consta de dos bloques principales. El primer bloque es responsable de contener la lógica combinacional, encargándose de ejecutar las operaciones requeridas. Mientras que el segundo bloque se encarga de registrar los resultados obtenidos a partir de la lógica combinacional.

Es importante destacar que la ALU solo posee un operando, lo que significa que todas las operaciones se realizan utilizando el registro ACC como el segundo operando. Además, el registro ACC también almacenará el resultado de la operación realizada.

Por ejemplo, si deseamos efectuar una operación AND, esta se calcula tomando como entrada el operando "op" junto con el valor actual almacenado en el registro ACC. Después de realizar el cálculo de la operación AND, el resultado se registra nuevamente en el registro ACC.

El procedimiento para llevar a cabo la operación consiste en colocar el valor en la entrada "op" junto con el código correspondiente a la operación que se desea realizar (en este caso, el código para la operación AND es cero). Luego, se establece el enable en uno, de manera que, al ocurrir el flanco de clock, se registre el resultado de la operación en el registro ACC.

La entidad de la ALU es la siguiente:

entity alu is

generic (DATA_BITS: integer := 16);

port (clk : in std_logic;

rst : in std_logic;

ena : in std_logic;

code: in std_logic_vector (3 downto 0);

op : in std_logic_vector (DATA_BITS-1 downto 0);

acc : out std_logic_vector (DATA_BITS-1 downto 0);

zero : out std_logic;

overflow : out std_logic;

carryBorrow : out std_logic;

negative : out std_logic);

end alu;

Donde:

- clk: Es el clock del sistema
- rst: Es el reset del sistema
- ena: Habilita la operación de la alu. Al colocarse en 1
- code: Indica qué operación debe realizar
- carryIn: Es el carry/borrow de entrada
- carryOut: Es el carry/borrow de salida
- acc: Es el valor del registro acumulador.
- sat: Si vale uno el resultado de la operación se satura.
- zero: Se coloca en uno cuando la res vale cero
- overflow: Salida que indica si hubo overflow en la operación.
- negative: Indica si el resultado es negativo.

Tabla de operaciones a realizar.

Code	Operación	Descripción
0	$ACC = ACC \text{ and } op$	and bit a bit
1	$ACC = ACC \text{ or } op$	or bit a bit
2	$ACC = ACC \text{ xor } op$	xor bit a bit
3	$(carry, ACC) = ACC + op + carry$	Suma con carry in y carry out Si el la entrada sat está en uno se satura el resultado de la operación
4	$(borrow, ACC) = ACC - op + borrow$	Resta con borrow in y borrow out Si la entrada sat está en uno se satura el resultado de la operación
5	$(Carry, ACC) = (Carry, ACC) \ll op, \text{ Si } op \geq 0$ $(Carry, ACC) = (Carry, ACC) \gg op, \text{ Si } op < 0$	Rotación con carry (op vale entre -16 y 16)
6	$ACC = op$	Carga ACC con op
7	$carrySet = op$	Setea el carry in en 1 ó 0
8	$satSet = op$	Configura la saturación del acumulador al sumar o restar
9 a 15	Uso futuro	No ejecuta ninguna acción

Tabla de flags

Code	Operación	carry borrow	zero	negative	overflow
0	ACC = ACC and op	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
1	ACC = ACC or op	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
2	ACC = ACC xor op	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
3	(carry,ACC) = ACC + op + carry	Carry	1, si ACC es cero	1, si ACC es negativo	Overflow de la operación
4	(borrow,ACC) = ACC - op + borrow	borrow	1, si ACC es cero	1, si ACC es negativo	Overflow de la operación
5	(Carry, ACC) = (Carry, ACC) << op, Si op >= 0 (Carry, ACC) = (Carry, ACC) >> op, Si op < 0	Si op = 0, Carry sin cambio Si op > 0, Carry = ACC (16 - op) Si op < 0, Carry = ACC (-1 - op)	1, si ACC es cero	1, si ACC es negativo	Sin cambio
6	ACC = op	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
7	carrySet	Carry = op(0)	Sin cambio	Sin cambio	Sin cambio
----	satSet	Sin cambio	Sin cambio	Sin cambio	Sin cambio

■ ■ ■

16. Ejercicios integradores II

1. Implemente un transmisor de la UART (8N1).

entity *uartTx* is

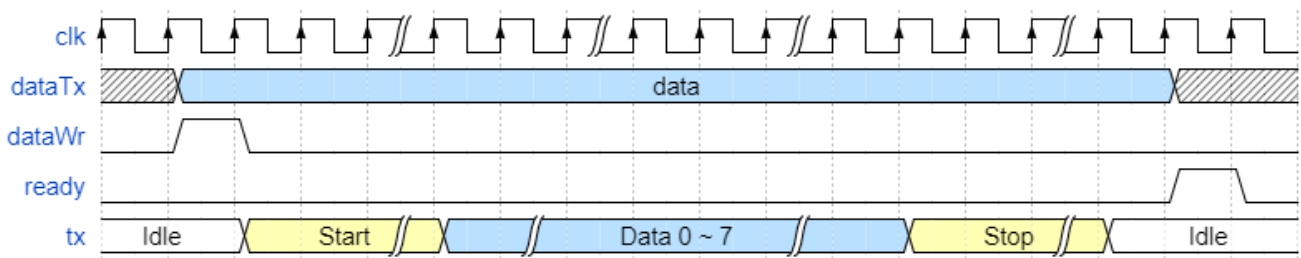
```
Generic (baudRate : integer := 9600;  
        sysClk    : integer := 100000000;  
        dataSize  : integer := 8);  
Port ( clk        : in std_logic;  
      rst         : in std_logic;  
      dataWr      : in std_logic;  
      dataTx      : in std_logic_vector (dataSize - 1 downto 0);  
      ready       : out std_logic;  
      tx          : out std_logic);
```

end *uartTx*;

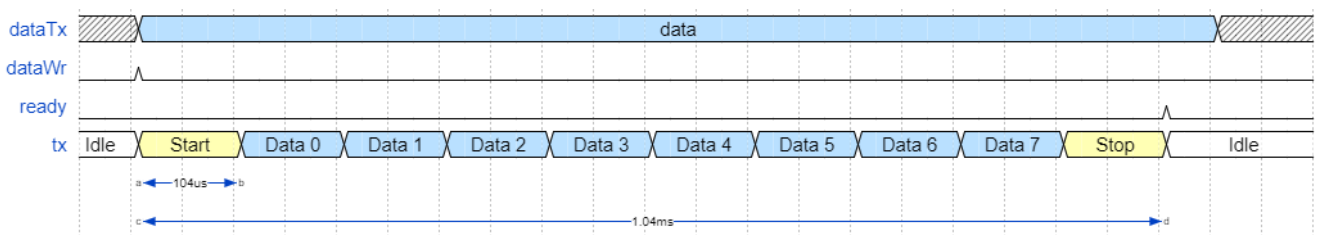
Donde:

- dataTx: Es el dato a Tx.
- dataWr: Es una señal que indica el inicio de la transmisión de los datos. Dura un pulso de clk
- ready: Indica cuando el tx está nuevamente disponible para Tx. Dura un pulso de clk
- tx: La salida de transmisión.
- sysClk: Es la frecuencia de clock del sistema

Ejemplo de Tx de datos con el clock como referencia.



Ejemplo de Tx de datos para observar el tiempo de transmisión de un bit y el byte entero. Las señales dataTx y ready se ponen como referencia, tenga en cuenta al observar el diagrama que duran un ciclo de clock del sistema.



El pin tx toma el valor cero en el bit start, uno en el bit stop y uno en el estado idle. El tiempo por bit es de 104us mientras que el tiempo de tx total es de 1.04ms a 9600 baudios.

Realice

- Diagrama de bloques y estados de lo implementado.
- Realice el diseño en VHDL
- Realice el testbench del diseño realizado suponiendo que el baud rate es de 1Mbaudios y el clock del sistema es de 100MHz

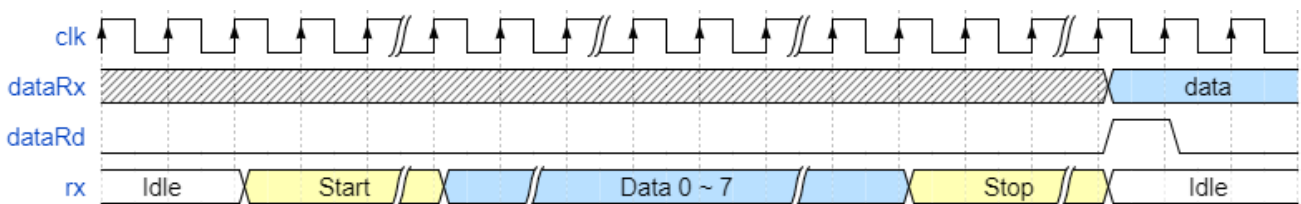
2. Implemente un receptor de la UART (8N1). Verifique el funcionamiento con una simulación *entity uartRx* is

```
Generic (baudRate : integer := 9600;
        sysClk    : integer := 100000000;
        dataSize  : integer := 8);
Port ( clk        : in std_logic;
      rst         : in std_logic;
      dataRd      : out std_logic;
      dataRx      : out std_logic_vector (dataSize - 1 downto 0);
      rx          : in std_logic);
end uartRx;
```

Donde:

- dataRx: Es el dato recibido.
- dataRd: Indica cuando se recibió un dato. Dura un pulso de clk.
- rx: Entrada de recepción.

El siguiente gráfico muestra la recepción de datos con el clk de referencia.



Realice

- Diagrama de bloques y estados de lo implementado.
- Realice el diseño en VHDL
- Realice el testbench del diseño realizado suponiendo que el baud rate es de 1Mbaudios y el clock del sistema es de 100MHz

3. Usando el módulo transmisor y receptor implementado anteriormente como componente arme una UART.

entity uart is

```
Generic (baudRate : integer := 9600;
        sysClk    : integer := 100000000;
        dataSize  : integer := 8);
Port ( clk        : in std_logic;
      rst         : in std_logic;
      dataWr      : in std_logic;
      dataTx      : in std_logic_vector (dataSize - 1 downto 0);
      ready       : out std_logic;
      tx          : out std_logic;
      dataRd      : out std_logic;
      dataRx      : out std_logic_vector (dataSize - 1 downto 0);
      rx          : in std_logic);
end uart;
```

Se pide:

- Diagrama de bloques y estados de lo implementado.
- Realice el diseño en VHDL
- Realice el testbench del diseño realizado suponiendo que el baud rate es de 1Mbaudios y el clock del sistema es de 100MHz

